R.M.K
GROUP OF ENGINEERING INSTITUTIONS



R.M.K
GROUP OF INSTITUTIONS











Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

R.M.K
GROUP OF
INSTITUTIONS

22CS702
DATA ANALYTICS

Department: IT
Batch/Year: 2023-2027 /III YEAR
Created by:
Ms. N. Arockia Rosy / Asst. Professor
Date: 14– 07 - 2025

R.M.K GROUP OF INSTITUTIONS

1. Table of Contents

S.NO	Topic	Page No.
1.	Contents	5
2.	Course Objectives	6
3.	Pre-Requisites	7
4.	Syllabus	8
5.	Course outcomes	9
6.	CO- PO/PSO Mapping	11
7.	Lecture Plan	13
8.	Activity based learning	14
9.	Lecture notes	16
10.	Assignments	134
11.	Part A Q & A	136
12.	Part B Qs	141
13.	Supportive online Certification courses	142
14.	Real time Applications in day to day life and to Industry	143
15	Contents beyond the Syllabus	145
16	Assessment Schedule	147
17	Prescribed Text Books and Reference Books	148
18	Mini Project Suggestions	149

R.M.K GROUP OF INSTITUTIONS

2. Course Objectives

- To explain the fundamentals of big data and data analytics
- To discuss the Hadoop framework
- To explain about exploratory data analysis and data manipulation tools
- To analyse and interpret streaming data
- To discuss various applications of data analytics

R.M.K GROUP OF INSTITUTIONS

3. Pre-Requisites

- 22CS102 – Problem Solving using C++
- 22CS303 – Design and Analysis of Algorithms
- 22IT401 – Artificial Intelligence and Machine Learning

R.M.K GROUP OF INSTITUTIONS

4.SYLLABUS

20CS702	DATA ANALYTICS	L T P C
		3 0 0 3

UNIT I INTRODUCTION 12

Evolution of Big Data-Definition of Big Data-Challenges with Big Data- Traditional Business Intelligence (BI) versus Big Data-Introduction to big data analytics-Classification of Analytics-Analytics Tools- Importance of big data analytics.

List of Exercise/Experiments:

1. Download, install and explore the features of R/Python for data analytics.
2. Working with Numpy arrays

UNIT II HADOOP FRAMEWORK 12

Introducing Hadoop- RDBMS versus Hadoop- Hadoop Overview-HDFS (Hadoop Distributed File System)- Processing Data with Hadoop- Managing Resources and Applications with Hadoop YARN - Interacting with Hadoop Ecosystem.

List of Exercise/Experiments:

1. Working with Pandas data frames
2. Basic plots using Matplotlib

UNIT III EXPLORATORY DATA ANALYSIS 12

EDA fundamentals – Understanding data science – Significance of EDA – Making sense of data – Comparing EDA with classical and Bayesian analysis – Software tools for EDA –Data transformation techniques - Introduction to NoSQL – MongoDB: RDBMS Vs MongoDB – Data Types – Query Language – Hive – Hive Architecture – Data Types – File Formats - Hive Query Language (HQL) – RC File Implementation – User Defined Functions.

List of Exercise/Experiments:

1. Statistical and Probability measures - Frequency distributions, Mean, Mode, Standard Deviation, Variability, Normal curves, Correlation and scatter plots, Correlation coefficient, Regression.
2. Use the standard benchmark data set for performing the following:

a) Univariate Analysis: Frequency, Mean, Median, Mode, Variance, Standard Deviation, Skewness and Kurtosis.
b) Bivariate Analysis: Linear and logistic regression modelling.
c) Multiple Regression Analysis
d) Compare the results of the above analysis for the two data sets

R.M.K GROUP OF INSTITUTIONS

4.SYLLABUS

20CS702	DATA ANALYTICS	L T P C
		3 0 0 3

UNIT IV MINING DATA STREAMS 12

The data stream model – stream queries-sampling data in a stream-general streaming problem- filtering streams-analysis of filtering- dealing with infinite streams- Counting Distance Elements in a Stream – Estimating Moments – Counting Ones in Window – Decaying Windows.

List of Exercise/Experiments:

1. Apply and explore various plotting functions on any data set.
2. Implement the following algorithms on real time stream data sets.

Support Vector Machine
Decision tree classifier
Clustering Algorithms

UNIT V APPLICATIONS 12

Application: Sales and Marketing – Industry Specific Data Mining – microRNA Data Analysis Case Study – Credit Scoring Case Study – Data Mining Nontabular Data.

List of Exercise/Experiments:

1. Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using the standard Heart Disease Data Set. You can use Java/Python ML library classes/API
2. Mini Project: The project should contain the following components

Realtime dataset
Data preparation & Transformation
Handling missing Data
Data Storage
Algorithm for data analytics
Data visualization: Charts, Heatmap, Crosstab, Treemap

R.M.K GROUP OF INSTITUTIONS

5. COURSE OUTCOMES

At the end of this course, the students will be able to:

COURSE OUTCOMES		HKL
CO1	Understand the basics of big data, including its definition, challenges, and differences from traditional BI.	K2
CO2	Use R and Python to analyze data sets and perform basic statistical analyses and visualizations.	K3
CO3	Compare the functionalities of RDBMS and Hadoop in data processing and resource management.	K4
CO4	Conduct exploratory data analysis (EDA) and understand its role compared to classical and Bayesian analysis.	K4
CO5	Implement various data stream mining algorithms for real-time data analysis.	K3
CO6	Develop real-world applications of big data analytics, covering data preparation, storage, analytics, and visualization	K5

R.M.K GROUP OF INSTITUTIONS

CO – PO/PSO Mapping

R.M.K
GROUP OF
INSTITUTIONS

6.CO – PO /PSO Mapping Matrix

CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PS O1	PS O2	PS 03
1	3	2	2	2	1	2	1	1	1	1	1	1	2	2	2
2	3	3	3	2	3	2	1	1	1	1	1	1	2	2	2
3	3	3	3	2	3	2	1	3	3	1	1	3	2	2	2
4	3	3	3	2	3	2	1	3	3	1	1	3	2	2	2
5	3	3	3	2	3	2	1	3	3	1	1	3	2	2	2
6	3	3	3	2	3	2	1	3	3	1	1	3	2	2	2

R.M.K GROUP OF INSTITUTIONS

Lecture Plan

Unit - III

LECTURE PLAN – Unit 3- EXPLORATORY DATA ANALYSIS

Sl. No.	Topic	Number of Periods	Proposed Date	Actual Lecture Date	CO	Taxonomy Level	Mode of Delivery
1	EDA fundamentals, Understanding data science	1			3	K2	Chalk & Talk
2	Significance of EDA, Making sense of data	1			3	K2	Chalk & Talk
3	Comparing EDA with classical and Bayesian analysis, Software tools for EDA	1			3	K2	Chalk & Talk
4	Data transformation techniques, Introduction to NoSQL	1			3	K2	Chalk & Talk
5	MongoDB: RDBMS Vs MongoDB, Data Types	1			3	K2	Chalk & Talk
6	Query Language, Hive	1			3	K2	Chalk & Talk
7	Hive Architecture, Data Types	1			3	K2	Chalk & Talk
8	File Formats, Hive Query Language (HQL)	1			3	K2	Chalk & Talk
9	RC File Implementation, User Defined Functions	1			3	K2	Chalk & Talk

8. ACTIVITY BASED LEARNING

Implement EDA process using Automobile Predictive analysis Dataset.

- Understanding the given dataset and EDA helps clean up the given dataset.
- It gives you a clear picture of the features and the relationships between them.
- Providing guidelines for essential variables and leaving behind/removing non-essential variables.
- Handling Missing values or human error.
- Identifying outliers.
- EDA process would be maximizing insights of a dataset.

Guidelines to do an activity :

1) Students can form group. (3 students / team)

1) Import necessary packages.

1)Conduct Peer review. (each team will be reviewed by all other teams and mentors)

Useful link:

https://www.analyticsvidhya.com/blog/2021/04/rapid-fire-eda-process-using-python-for-ml-implementation/

R.M.K GROUP OF INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

UNIT-III

EXPLORATORY DATA ANALYSIS

R.M.K
GROUP OF
INSTITUTIONS

9.LECTURE NOTES

1. EDA fundamentals

- Data encompasses a collection of discrete objects, numbers, words, events, facts, measurements, observations, or even descriptions of things. Such data is collected and stored by every event or process occurring in several disciplines, including biology, economics, engineering, marketing, and others.

- Processing such data provides useful information and processing such information generates useful knowledge. But an important question is: how can we generate meaningful and useful information from such data? An answer to this question is EDA.

- EDA is a process of examining the available dataset to discover patterns, spot anomalies, test hypotheses, and check assumptions using statistical measures
.
- As mentioned in several studies, the primary aim of EDA is to examine what data can tell us before actually going through formal modeling or hypothesis formulation. John Tuckey promoted EDA to statisticians to examine and discover the data and create newer hypotheses that could be used for the development of a newer approach in data collection and experimentations.

2. Understanding data science

- Data science is at the peak of its hype and the skills for data scientists are changing. Now, data scientists are not only required to build a performant model, but it is essential for them to explain the results obtained and use the result for business intelligence.

- Data science involves cross-disciplinary knowledge from computer science, data, statistics, and mathematics.
- There are several phases of data analysis, including data requirements, data collection, data processing, data cleaning, exploratory data analysis, modeling and algorithms, and data product and communication.
- These phases are similar to the CRoss-Industry Standard Process for data mining (CRISP) framework in data mining.
- The main takeaway here is the stages of EDA, as it is an important aspect of data analysis and data mining. Let's understand in brief what these stages are:

i. Data requirements: There can be various sources of data for an organization. It is important to comprehend what type of data is required for the organization to be collected, curated, and sto

- For example, an application tracking the sleeping pattern of patients suffering from dementia requires several types of sensors' data storage, such as sleep data, heart rate from the patient, electro-dermal activities, and user activities
- All of these data points are required to correctly diagnose the mental state of the person. Hence, these are mandatory requirements for the application.
- In addition to this, it is required to categorize the data, numerical or categorical, and the format of storage and dissemination.

ii. Data collection: Data collected from several sources must be stored in the correct format and transferred to the right information technology personnel within a company. As mentioned previously, data can be collected from several objects on several events using different types of sensors and storage tools.

R.M.K GROUP OF INSTITUTIONS

iii. **Data processing:** Preprocessing involves the process of pre-curating the dataset before actual analysis.

- Common tasks involve correctly exporting the dataset, placing them under the right tables, structuring them, and exporting them in the correct format.

iv. **Data cleaning:** Preprocessed data is still not ready for detailed analysis. It must be correctly transformed for an incompleteness check, duplicates check, error check, and missing value check.

- These tasks are performed in the data cleaning stage, which involves responsibilities such as matching the correct record, finding inaccuracies in the dataset, understanding the overall data quality, removing duplicate items, and filling in the missing values.
- However, how could we identify these anomalies on any dataset? Finding such data issues requires us to perform some analytical
- To understand briefly, data cleaning is dependent on the types of data under
- Hence, it is most essential for data scientists or EDA experts to comprehend different types of datasets. An example of data cleaning would be using outlier detection methods for quantitative data cleaning.

v. **EDA:** Exploratory data analysis is the stage where we actually start to understand the message contained in the data. It should be noted that several types of data transformation techniques might be required during the process of exploration.

v. **Modeling and algorithm:** From a data science perspective, generalized models or mathematical formulas can represent or exhibit relationships among different variables, such as correlation or causation.

- These models or equations involve one or more variables that depend on other variables to cause an event.
- For example, when buying, say, pens, the total price of pens(Total) = price for one pen(UnitPrice) * the number of pens bought (Quantity).
- Hence, our model would be Total = UnitPrice * Quantity. Here, the total price is dependent on the unit price.
- Hence, the total price is referred to as the dependent variable and the unit price is referred to as an independent variable.
- In general, a model always describes the relationship between independent and dependent variables. Inferential statistics deals with quantifying relationships between particular variables.
- The Judd model for describing the relationship between data, model, and error still holds true: Data = Model + Error.

vii. Data Product: Any computer software that uses data as inputs, produces outputs, and provides feedback based on the output to control the environment is referred to as a data

pr
- A data product is generally based on a model developed during data analysis, for example, a recommendation model that inputs user purchase history and recommends a related item that the user is highly likely to buy.

viii. Communication: This stage deals with disseminating the results to end stakeholders to use the result for business intelligence.

- One of the most notable steps in this stage is data visualization. Visualization deals with information relay techniques such as tables, charts, summary diagrams, and bar charts to show the analyzed result.

3. Significance of EDA

- Different fields of science, economics, engineering, and marketing accumulate and store data primarily in electronic databases. Appropriate and well-established decisions should be made using the data collected.
- It is practically impossible to make sense of datasets containing more than a handful of data points without the help of computer programs. To be certain of the insights that the collected data provides and to make further decisions, data mining is performed where we go through distinctive analysis processes.
- Exploratory data analysis is key, and usually the first exercise in data mining. It allows us to visualize data to understand it as well as to create hypotheses for further analysis. The exploratory analysis centers around creating a synopsis of data or insights for the next steps in a data mining project.
- EDA actually reveals ground truth about the content without making any underlying assumptions. This is the fact that data scientists use this process to actually understand what type of modeling and hypotheses can be
- Key components of exploratory data analysis include summarizing data, statistical analysis, and visualization of data.
- Python provides expert tools for exploratory analysis, with pandas for summarizing; scipy, along with others, for statistical analysis; and matplotlib and plotly for visualizations.
- After understanding the significance of EDA, let's discover what are the most generic steps involved in EDA

R.M.K GROUP OF INSTITUTIONS

❖ Steps in EDA

- Having understood what EDA is, and its significance, let's understand the various steps involved in data analysis. Basically, it involves four different steps. Let's go through each of them to get a brief understanding of each step:

1. **Problem definition:** Before trying to extract useful insight from the data, it is essential to define the business problem to be solved.

- The problem definition works as the driving force for a data analysis plan execution.
- The main tasks involved in problem definition are defining the main objective of the analysis, defining the main deliverables, outlining the main roles and responsibilities, obtaining the current status of the data, defining the timetable, and performing cost/benefit analysis. Based on such a problem definition, an execution plan can be created.

2. **Data preparation:** This step involves methods for preparing the before actual an

- In this step, we define the sources of data, define data schemas and tables, understand the main characteristics of the data, clean the dataset, delete non-relevant datasets, transform the data, and divide the data into required chunks for analysis.

3. **Data analysis:** This is one of the most crucial steps that deals with descriptive statistics and analysis of the data.

- The main tasks involve summarizing the data, finding the hidden correlation and relationships among the data, developing predictive models, evaluating the models, and calculating the accuracies.

R.M.K GROUP OF INSTITUTIONS

- Some of the techniques used for data summarization are summary tables, graphs, descriptive statistics, inferential statistics, correlation statistics, searching, grouping, and mathematical models.

4. **Development and representation of the results:** This step involves presenting the dataset to the target audience in the form of graphs, summary tables, maps, and diagrams.
    - This is also an essential step as the result analyzed from the dataset should be interpretable by the business stakeholders, which is one of the major goals of EDA.
    - Most of the graphical analysis techniques include scattering plots, character plots, histograms, box plots, residual plots, mean plots, and others.

IV. Making sense of data

- It is crucial to identify the type of data under analysis. We will learn about different types of data that is encountered during
- Different disciplines store different kinds of data for different purposes. For example, medical researchers store patients' data, universities store students' and teachers' data, and real estate industries storehouse and building datasets.
- A dataset contains many observations about a particular object. For instance, a dataset about patients in a hospital can contain many observations.
- A patient can be described by a patient identifier (ID), name, address, weight, date of birth, address, email, and gender. Each of these features that describes a patient is a variable. Each observation can have a specific value for each of these variables.

R.M.K GROUP OF INSTITUTIONS

For example, a patient can have the following:

```
PATIENT_ID = 1001
Name = Yoshmi Mukhiya
Address = Mannsverk 61, 5094, Bergen, Norway
Date of birth = 10th July 2018
Email = yoshmimukhiya@gmail.com
Weight = 10
Gender = Female
```

- These datasets are stored in hospitals and are presented for analysis. Most of this data is stored in some sort of database management system in tables/schema. An example of a table for storing patient information is shown here:

PATIENT_ID	NAME	ADDRESS	DOB	EMAIL	Gender	WEIGHT
001	Suresh Kumar Mukhiya	Mannsverk, 61	30.12.1989	skmu@hvl.no	Male	68
002	Yoshmi Mukhiya	Mannsverk 61, 5094, Bergen	10.07.2018	yoshmimukhiya@gmail.com	Female	1
003	Anju Mukhiya	Mannsverk 61, 5094, Bergen	10.12.1997	anjumukhiya@gmail.com	Female	24
004	Asha Gaire	Butwal, Nepal	30.11.1990	aasha.gaire@gmail.com	Female	23
005	Ola Nordmann	Danmark, Sweden	12.12.1789	ola@gmail.com	Male	75

- To summarize the preceding table, there are four observations (001, 002, 003, 004, 005). Each observation describes variables (PatientID, name, address, dob, email, gender, and weight).
- Most of the dataset broadly falls into two groups—numerical data and categorical data.

1. Numerical data

- This data has a sense of measurement involved in it; for example, a person's age, height, weight, blood pressure, heart rate, temperature, number of teeth, number of bones, and the number of family members. This data is often referred to as quantitative data in statistics.
- The numerical dataset can be either discrete or continuous types.

1. Discrete data

- This is data that is countable and its values can be listed out. For example, if we flip a coin, the number of heads in 200 coin flips can take values from 0 to 200 (finite) cases.
- A variable that represents a discrete dataset is referred to as a discrete variable. The discrete variable takes a fixed number of distinct
- For example, the Country variable can have values such as Nepal, India, Norway, and Japan. It is fixed. The Rank variable of a student in a classroom can take values from 1, 2, 3, 4, 5, and so on.

2. Continuous data

- A variable that can have an infinite number of numerical values within a specific range is classified as continuous data.
- A variable describing continuous data is a continuous variable. For example, what is the temperature of your city today? Can we be finite?
- Similarly, the weight variable is also a continuous variable.

2. Categorical data

- ➢ This type of data represents the characteristics of an object; for example, gender, marital status, type of address, or categories of the movies. This data is often referred to as qualitative datasets in statistics.
- ➢ Some of the most common types of categorical data you can find in data:

- Gender (Male, Female, Other, or Unknown)
- Marital Status (Divorced, Legally Separated, Married, Never Married, Unmarried, Widowed, or Unknown)
- Movie genres (Action, Adventure, Comedy, Crime, Drama, Fantasy, Historical, Horror, Mystery, Philosophical, Political, Saga, Satire, Science Fiction, Social, Thriller, Urban, or Western)
- Blood type (A, B, AB, or
- Types of drugs (Stimulants, Depressants, Hallucinogens, Dissociatives, Opioids, Inhalants, or Cannabis)

A variable describing categorical data is referred to as a categorical variable.

These types of variables can have one of a limited number of values.

There are different types of categorical variables:

- A binary categorical variable can take exactly two values and is also referred to as a **dichotomous variable**. For example, when we create an experiment, the result is either success or failure. Hence, results can be understood as a **binary categorical variable**.
- Polytomous variables are categorical variables that can take more than two possible values. For example, marital status can have several values, such as divorced, legally separated, married, never married, unmarried, widowed and unknown. Since marital status can take more

than two possible values, it is a polytomous variable.

- Most of the categorical dataset follows either nominal or ordinal measurement scales.

1. Measurement scales

There are four different types of measurement scales described in statistics: nominal, ordinal, interval, and ratio. These scales are used more in academic industries. Let's understand each of them with some examples.

1. Nominal

These are practiced for labeling variables without any quantitative value. The scales are generally referred to as labels. And these scales are mutually exclusive and do not carry any numerical importance. Let's see some examples:

- What is your g
  - Mal
  - Femal
  - I prefer not to
  - Othe er

Other examples include the followin

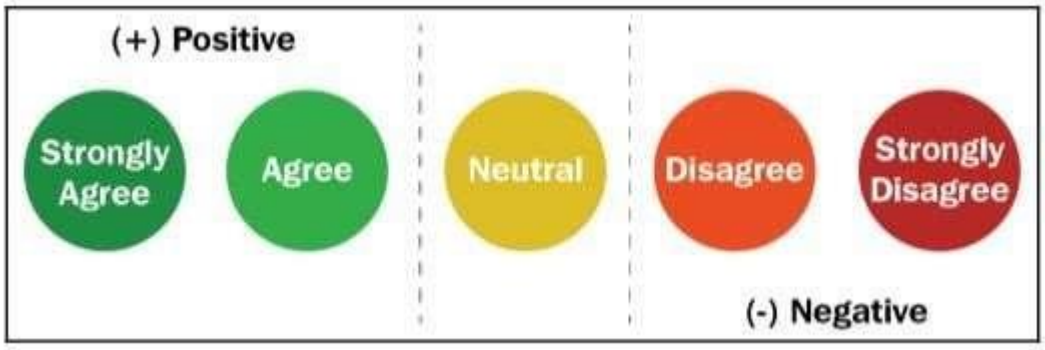
- The languages that are spoken in a particular country
- Biological species
- Parts of speech in grammar (noun, pronoun, adjective, and so on)
- Taxonomic ranks in biology (Archea, Bacteria, and Eukarya)

- Nominal scales are considered qualitative scales and the measurements that are taken using qualitative scales are considered qualitative data.

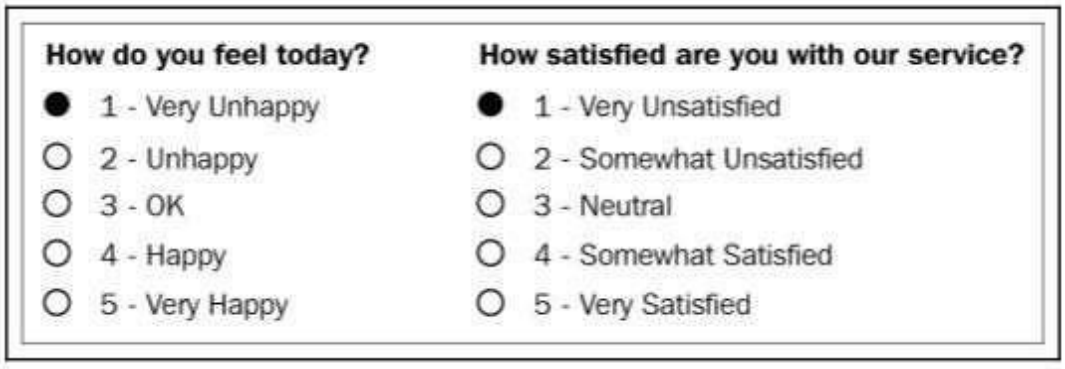
R.M.K GROUP OF INSTITUTIONS

4.2.1.2 Ordinal

- The main difference in the ordinal and nominal scale is the order. In ordinal scales, the order of the values is a significant factor.
- Let's check an example of ordinal scale using the Likert scale: WordPress is making content managers' lives easier. How do you feel about this statement? The following diagram shows the Likert scale:



- As depicted in the preceding diagram, the answer to the question of WordPress is making content managers' lives easier is scaled down to five different ordinal values, Strongly Agree, Agree, Neutral, Disagree, and Strongly
- Scales like these are referred to as the Likert scale. Similarly, the following diagram shows more examples of the Likert scale:



R.M.K GROUP OF INSTITUTIONS

- To make it easier, consider ordinal scales as an order of ranking (1st, 2nd, 3rd, 4th, and so on). The median item is allowed as the measure of central tendency; however, the average is not permitted.

3. Interval

- In interval scales, both the order and exact differences between the values are significant.
- Interval scales are widely used in statistics, for example, in the measure of central tendencies—mean, median, mode, and standard deviations.
- Examples include location in Cartesian coordinates and direction measured in degrees from magnetic north. The mean, median, and mode are allowed on interval data.

4. Ratio

- Ratio scales contain order, exact values, and absolute zero, which makes it possible to be used in descriptive and inferential
- These scales provide numerous possibilities for statistical
- Mathematical operations, the measure of central tendencies, and the measure of dispersion and coefficient of variation can also be computed from such scales.
- Examples include a measure of energy, mass, length, duration, electrical energy, plan angle, and volume. The following table gives a summary of the data types and scale measures:

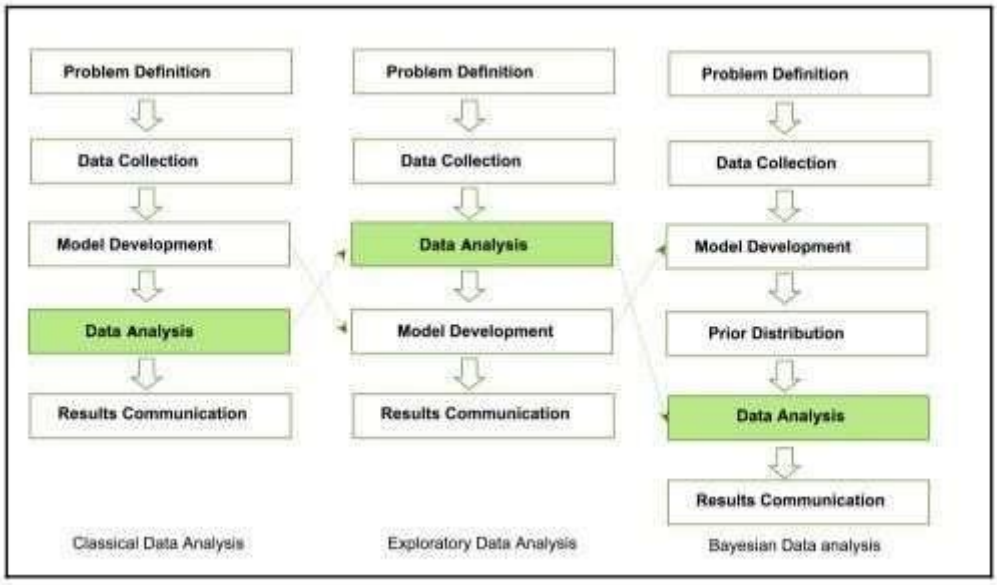
Provides:	Nominal	Ordinal	Interval	Ratio
The "order"of values is known		✓	✓	✓
"Counts," aka "Frequency of Distribution"	✓	✓	✓	✓
Mode	✓	✓	✓	✓
Median		✓	✓	✓
Mean			✓	✓
Can quantify the difference between each value			✓	✓
Can add or subtract values			✓	✓
Can multiple and divide values				✓
Has "true zero"				✓

5. Comparing EDA with classical and Bayesian analysis

There are several approaches to data analysis. The most popular ones that are the following:

- **Classical data analysis:** For the classical data analysis approach, the problem definition and data collection step are followed by model development, which is followed by analysis and result
- **Exploratory data analysis approach:** For the EDA approach, it follows the same approach as classical data analysis except the model imposition and the data analysis steps are swapped. The main focus is on the data, its structure, outliers, models, and visualizations. Generally, in EDA, we do not impose any deterministic or probabilistic models on the data.
- **Bayesian data analysis approach:** The Bayesian approach incorporates prior probability distribution knowledge into the analysis steps as shown in the following diagram. The prior probability distribution of any quantity expresses the belief about that particular quantity before considering some evidence.

The following diagram shows three different approaches for data analysis illustrating the difference in their execution steps:



- Data analysts and data scientists freely mix steps mentioned in the preceding approaches to get meaningful insights from the
- In addition to that, it is essentially difficult to judge or estimate which model is best for data analysis. All of them have their paradigms and are suitable for different types of data analysis.

6. Software tools available for EDA

- There are several software tools that are available to facilitate EDA. Here, we are going to outline some of the open source tools:
- **Python:** This is an open source programming language widely used in data analysis, data mining, and data science (https://www.python.org/).
- **R programming language:** R is an open source programming language that is widely utilized in statistical computation and graphical data analysis (https:// www.r-project.org).

- **Weka:** This is an open source data mining package that involves several EDA tools and algorithms (https://www.cs.waikato.ac.nz/ml/weka/).
- **KNIME:** This is an open source tool for data analysis and is based on Eclipse (https://www.knime.com/).

7. Data transformation techniques

1. Performing data deduplication

It is very likely that our dataframe contains duplicate rows. Removing them is essential to enhance the quality of the dataset. This can be done with the following steps:

1. Let's consider a simple dataframe, as follows:

```
frame3 = pd.DataFrame({'column 1': ['Looping'] * 3 + ['Functions']
* 4, 'column 2': [10, 10, 22, 23, 23, 24, 24]})
frame3
```

The preceding code creates a simple dataframe with two columns. From the below screenshot, In both columns, there are some duplicate entries:

	column 1	column 2
0	Looping	10
1	Looping	10
2	Looping	22
3	Functions	23
4	Functions	23
5	Functions	24
6	Functions	24

2. The pandas dataframe comes with a duplicated() method that returns a Boolean series stating which of the rows are duplicates:

```
frame3.duplicated()
```

The output of the preceding code is easy to interpret:

```
0    False
1     True
2    False
3    False
4     True
5    False
6     True
dtype: bool
```

The rows that say True are the ones that contain duplicated data.

3. Now, we can drop these duplicates using the drop_duplicates() method:

```
frame4 = frame3.drop_duplicates()
frame4
```

The output of the preceding code is as follows:

	column 1	column 2
0	Looping	10
2	Looping	22
3	Functions	23
5	Functions	24

Note that rows 1, 4, and 6 are removed.

R.M.K GROUP OF INSTITUTIONS

- Basically, both the duplicated() and drop_duplicates() methods consider all of the columns for comparison. Instead of all the columns, we could specify any subset of the columns to detect duplicated items.

4. Let's add a new column and try to find duplicated items based on the second column:

```
frame3['column 3'] = range(7)
frame5 = frame3.drop_duplicates(['column 2'])
frame5
```

The output of the preceding snippet is as follows:

	column 1	column 2	column 3
0	Looping	10	0
2	Looping	22	2
3	Functions	23	3
5	Functions	24	5

- Note that both the duplicated and drop_duplicates methods keep the first observed value during the duplication removal process. If we pass the take_last=True argument, the methods return the last one.

7.2 Replacing values

Often, it is essential to find and replace some values inside a dataframe. This can be done with the following steps:

R.M.K GROUP OF INSTITUTIONS

1. We can use the replace method in such cases:

```
import numpy as np
replaceFrame = pd.DataFrame({'column 1': [200., 3000., -786.,
3000., 234., 444., -786., 332., 3332. ], 'column 2': range(9)})
replaceFrame.replace(to_replace =-786, value= np.nan)
```

The output of the preceding code is as follows:

	column 1	column 2
0	200.0	0
1	3000.0	1
2	NaN	2
3	3000.0	3
4	234.0	4
5	444.0	5
6	NaN	6
7	332.0	7
8	3332.0	8

Note that we just replaced one value with the other values. We can also replace multiple values at once.

2. In order to do so, we display them using a list:

```
replaceFrame = pd.DataFrame({'column 1': [200., 3000., -786.,
3000., 234., 444., -786., 332., 3332. ], 'column 2': range(9)})
replaceFrame.replace(to_replace =[-786, 0], value= [np.nan, 2])
```

In the preceding code, there are two replacements. All -786 values will be replaced by NaN and all 0 values will be replaced by 2.

R.M.K GROUP OF INSTITUTIONS

7.3 Handling missing data

Whenever there are missing values, a NaN value is used, which indicates that there is no value specified for that particular index. There could be several reasons why a value could be NaN:

- It can happen when data is retrieved from an external source and there are some incomplete values in the dataset.
- It can also happen when we join two different datasets and some values are not matched.
- Missing values due to data collection errors.
- When the shape of data changes, there are new additional rows or columns that are not determined.
- Reindexing of data can result in incomplete data.

Let's see how we can work with the missing d

1. Let's assume we have a dataframe as sl

```
data = np.arange(15, 30).reshape(5, 3)
dfx = pd.DataFrame(data, index=['apple', 'banana', 'kiwi',
'grapes', 'mango'], columns=['store1', 'store2', 'store3'])
dfx
```

And the output of the preceding code is as follows:

	store1	store2	store3
apple	15	16	17
banana	18	19	20
kiwi	21	22	23
grapes	24	25	26
mango	27	28	29

Assume we have a chain of fruit stores all over town. Currently, the dataframe is showing sales of different fruits from different stores. None of the stores are reporting missing values.

2. Let's add some missing values to our dataframe: dfx['store4'] = np.nan
dfx.loc['watermelon'] = np.arange(15, 19) dfx.loc['oranges'] = np.nan
dfx['store5'] = np.nan
dfx['store4']['apple'] = 20.
dfx

And the output will now look like the following screenshot:

	store1	store2	store3	store4	store5
apple	15.0	16.0	17.0	20.0	NaN
banana	18.0	19.0	20.0	NaN	NaN
kiwi	21.0	22.0	23.0	NaN	NaN
grapes	24.0	25.0	26.0	NaN	NaN
mango	27.0	28.0	29.0	NaN	NaN
watermelon	15.0	16.0	17.0	18.0	NaN
oranges	NaN	NaN	NaN	NaN	NaN

- Note that we've added two more stores, store4 and store5, and two more types of fruits, watermelon and
- Assume that we know how many kilos of apples and watermelons were sold from store4, but we have not collected any data from store5. Moreover, none of the stores reported sales of oranges.

The following characteristics of missing values in the preceding dataframe are:

- An entire row can contain NaN values.
- An entire column can contain NaN values.
- Some (but not necessarily all) values in both a row and a column can be NaN.

Based on these characteristics, let's examine NaN values

7.3.1 NaN values in pandas objects

We can use the isnull() function from the pandas library to identify NaN va1l.ues: Check the following

example: dfx.isnull()

The output of the preceding code is as follows:

	store1	store2	store3	store4	store5
apple	False	False	False	False	True
banana	False	False	False	True	True
kiwi	False	False	False	True	True
grapes	False	False	False	True	True
mango	False	False	False	True	True
watermelon	False	False	False	False	True
oranges	True	True	True	True	True

Note that the True values indicate the values that are NaN. Alternatively, we can also use the notnull() method to do the same thing. The only difference would be that the function will indicate True for the values which are not null.

2. Check it out in act dfx.notnull(

And the output of this is as fo¨

	store1	store2	store3	store4	store5
apple	True	True	True	True	False
banana	True	True	True	False	False
kiwi	True	True	True	False	False
grapes	True	True	True	False	False
mango	True	True	True	False	False
watermelon	True	True	True	True	False
oranges	False	False	False	False	False

Compare these two tables. These two functions, notnull() and isnull(), are the complement to each other.

R.M.K GROUP OF INSTITUTIONS

3. We can use the sum() method to count the number of NaN values in each store. Check the following

code: dfx.isnull().sum()

And the output of the preceding code is as follows:

```
store1 1
store2 1
store3 1
store4 5
store5 7
dtype: int64
```

The fact that True is 1 and False is 0 is the main logic for summing. The preceding results show that one value was not reported by store1, store2, and store3. Five values were not reported by store4 and seven values were not reported by store5.

4. We can go one level deeper to find the total number of missing

dfx.isnull().sum().sum(

And the output of the preceding code is as follows:

15

This indicates 15 missing values in our stores. We can use an alternative way to find how many values were actually reported.

5. So, instead of counting the number of missing values, we can count the number of reported values:

dfx.count()

And the output of the preceding code is as follows:

```
store1 6
store2 6
store3 6
store4 2
store5 0
dtype: int64
```

We now know two different ways to find the missing values, and also how to count the missing values.

R.M.K GROUP OF INSTITUTIONS

7.3.2 Dropping missing values

One of the ways to handle missing values is to simply remove them from our dataset. We have seen that we can use the isnull() and notnull() functions from the pandas library to determine null values:

```
dfx.store4[dfx.store4.notnull()]
```

The output of the preceding code is as follows:

```
apple 20.0
watermelon 18.0
Name: store4, dtype: float64
```

The output shows that store4 only reported two items of data. Now, we can use the dropna() method to remove the rows:

```
dfx.store4.dropna(
)
```

The output of the preceding code is as follows:

```
apple 20.0
watermelon 18.0
Name: store4, dtype: float64
```

Note that the dropna() method just returns a copy of the dataframe by dropping the rows with NaN. The original dataframe is not changed.

- If dropna() is applied to the entire dataframe, then it will drop all the rows from the dataframe, because there is at least one NaN value in our dataframe:

```
dfx.dropna()
```

The output of the preceding code is an empty dataframe.

7.3.2.1 Dropping by rows

We can also drop rows that have NaN values. To do so, we can use the how=all argument to drop only those rows entire values are entirely

NaN: dfx.dropna(how='all')

The output of the preceding code is as follows:

	store1	store2	store3	store4	store5
apple	15.0	16.0	17.0	20.0	NaN
banana	18.0	19.0	20.0	NaN	NaN
kiwi	21.0	22.0	23.0	NaN	NaN
grapes	24.0	25.0	26.0	NaN	NaN
mango	27.0	28.0	29.0	NaN	NaN
watermelon	15.0	16.0	17.0	18.0	NaN

Note that only the orange rows are removed because those entire rows contained NaN values.

7.3.2.2 Dropping by columns

Furthermore, we can also pass axis=1 to indicate a check for NaN by columns.

Check the following exam dfx.dropna(how='all', axis=1) And the output of the preceding code is as follows:

	store1	store2	store3	store4
apple	15.0	16.0	17.0	20.0
banana	18.0	19.0	20.0	NaN
kiwi	21.0	22.0	23.0	NaN
grapes	24.0	25.0	26.0	NaN
mango	27.0	28.0	29.0	NaN
watermelon	15.0	16.0	17.0	18.0
oranges	NaN	NaN	NaN	NaN

R.M.K GROUP OF INSTITUTIONS

- Note that store5 is dropped from the dataframe.
- By passing in axis=1, we are instructing pandas to drop columns if all the values in the column are NaN.
- Furthermore, we can also pass another argument, thresh, to specify a minimum number of NaNs that must exist before the column should be dropped:

```
dfx.dropna(thresh=5, axis=1)
```

And the output of the preceding code is as follows:

	store1	store2	store3
apple	15.0	16.0	17.0
banana	18.0	19.0	20.0
kiwi	21.0	22.0	23.0
grapes	24.0	25.0	26.0
mango	27.0	28.0	29.0
watermelon	15.0	16.0	17.0
oranges	NaN	NaN	NaN

Compared to the preceding, note that even the store4 column is now dropped because it has more than five NaN values.

7.3.3 Mathematical operations with NaN

The pandas and numpy libraries handle NaN values differently for mathematical operations.

Consider the following example:

```
ar1 = np.array([100, 200, np.nan, 300])
ser1 = pd.Series(ar1)
ar1.mean(), ser1.mean()
```

The output of the preceding code is the following: (nan, 200.0)

Note the following things:

- When a NumPy function encounters NaN values, it returns NaN.
- Pandas, on the other hand, ignores the NaN values and moves ahead with processing. When performing the sum operation, NaN is treated as 0. If all the values are NaN, the result is also NaN.

Let's compute the total quantity of fruits sold by store4: ser2 = dfx.store4

ser2.sum()

The output of the preceding code is as follows: 38.0

Note that store4 has five NaN values. However, during the summing process, these values are treated as 0 and the result is 38.0.

Similarly, we can compute averages as shown here: ser2.mean(

The output of the code is the following:.0

Note that NaNs are treated as 0s. It is the same for cumulative summing: ser2.cumsum()

And the output of the preceding code is as follows:

apple 20.0
banana NaN
kiwi NaN
grapes NaN
mango NaN

R.M.K GROUP OF INSTITUTIONS

watermelon 38.0

oranges NaN

Name: store4, dtype: float64

Note that only actual values are affected in computing the cumulative sum.

7.3.4 Filling missing values

We can use the fillna() method to replace NaN values with any particular values.

Check the following example:

```
filledDf = dfx.fillna(0)
filledDf
```

The output of the preceding code is shown in the following screenshot:

	store1	store2	store3	store4	store5
apple	15.0	16.0	17.0	20.0	0.0
banana	18.0	19.0	20.0	0.0	0.0
kiwi	21.0	22.0	23.0	0.0	0.0
grapes	24.0	25.0	26.0	0.0	0.0
mango	27.0	28.0	29.0	0.0	0.0
watermelon	15.0	16.0	17.0	18.0	0.0
oranges	0.0	0.0	0.0	0.0	0.0

Note that in the preceding dataframe, all the NaN values are replaced by 0. Replacing the values with 0 will affect several statistics including mean, sum, and median.

Check the difference in the following two examples: dfx.mean()

And the output of the preceding code is as follows: store1 20.0

store2 21.0

store3 22.0

store4 19.0

store5 NaN

dtype: float64

Now, let's compute the mean from the filled dataframe with the following command:

```
filledDf.mean()
```

And the output we get is as follows:

store1 17.142857

store2 18.000000

store3 18.857143

store4 5.428571

store5 0.000000

dtype: float64

Note that there are slightly different values. Hence, filling with 0 might not be the optimal solution.

7.3.5 Backward and forward filling

NaN values can be filled based on the last known values. To understand this, let's consider taking our store dataframe as an

We want to fill store4 using the forward-filling technique:

```
df1.store4.fillna(method='ffill')
```

And the output of the preceding code is the following: apple 20.0

banana 20.0

kiwi 20.0

grapes 20.0

mango 20.0

watermelon 18.0

oranges 18.0

Name: store4, dtype: float64

R.M.K GROUP OF INSTITUTIONS

Here, from the forward-filling technique, the last known value is 20 and hence the rest of the NaN values are replaced by it.

The direction of the fill can be changed by changing method='bfill'. Check the following example:

```
dfx.store4.fillna(method='bfill')
```

And the output of the preceding code is as follows:

```
apple 20.0
banana 18.0
kiwi 18.0
grapes 18.0
mango 18.0
watermelon 18.0
oranges NaN
Name: store4, dtype: float64
```

Note here that the NaN values are replaced by 18.0.

7.3.6 Interpolating missing values

The pandas library provides the interpolate() function both for the series and the dataframe. By default, it performs a linear interpolation of our missing values.

Check the following examplsee:

```
r3 = pd.Series([100, np.nan, np.nan, np.nan, 292])
ser3.interpolate()
```

And the output of the preceding code is the following:

```
0 100.0
1 148.0
2 196.0
3 244.0
4 292.0
dtype: float64
```

R.M.K GROUP OF INSTITUTIONS

how these values are calculated?

- It is done by taking the first value before and after any sequence of the NaN values.
- In the preceding series, ser3, the first and the last values are 100 and 292 respectively. Hence, it calculates the next value as (292-100)/(5-1) = 48. So, the next value after 100 is 100 + 48 = 148.
- We can perform more complex interpolation techniques, especially with time series data.

7.4 Renaming axis indexes

Consider the example from the Reshaping and pivoting section. Say we want to transform the index terms to capital letters:

```
dframe1.index =
dframe1.index.map(str.upper)  dframe1
```

The output of the preceding code is as follows:

	Bergen	Oslo	Trondheim	Stavanger	Kristiansand
RAINFALL	0	1	2	3	4
HUMIDITY	5	6	7	8	9
WIND	10	11	12	13	14

Note that the indexes have been capitalized.

- If we want to create a transformed version of the dataframe, then we can use the rename() method.
- This method is handy when we do not want to modify the original data.

R.M.K GROUP OF INSTITUTIONS

Check the following example:

```
dframe1.rename(index=str.title,
columns=str.upper)
```

And the output of the code is as follows:

	BERGEN	OSLO	TRONDHEIM	STAVANGER	KRISTIANSAND
Rainfall	0	1	2	3	4
Humidity	5	6	7	8	9
Wind	10	11	12	13	14

The rename method does not make a copy of the dataframe.

7.5 Discretization and binning

Often when working with continuous datasets, we need to convert them into discrete or interval forms. Each interval is referred to as a bin, and hence the name binning comes into play:

1. Let's say we have data on the heights of a group of students as follows:

```
height = [120, 122, 125, 127, 121, 123, 137, 131, 161, 145, 141, 132]
```

And we want to convert that dataset into intervals of 118 to 125, 126 to 135, 136 to 160, and finally 160 and higher.

2. To convert the preceding dataset into intervals, we can use the cut() method provided by the pandas library:

```
bins = [118, 125, 135, 160, 200]
category = pd.cut(height,
bins) category
```

The output of the preceding code is as follows:

[(118, 125], (118, 125], (118, 125], (125, 135], (118, 125], ..., (125, 135], (160, 200], (135, 160], (135, 160], (125, 135]]

Length: 12 Categories (4, interval[int64]): [(118, 125] < (125, 135] < (135, 160] < (160, 200]]

R.M.K GROUP OF INSTITUTIONS

- A parenthesis indicates that the side is open.
- A square bracket means that it is closed or inclusive.

From the preceding code block, (118, 125] means the left-hand side is open and the right-hand side is closed.

This is mathematically denoted as follows:

$$(a, b] = \{x | a < x \leq b\}$$

Hence, 118 is not included, but anything greater than 118 is included, while 125 is included in the interval.

3. We can set a right=False argument to change the form of interval:

```
category2 = pd.cut(height, [118, 126, 136, 161, 200], right=False)
category2
```

And the output of the preceding code is as

[[118, 126), [118, 126), [118, 126), [126, 136), [118, 126), ..., [126, 136), [161, 200), [136, 161), [136, 161), [126, interval[int64]): [[118, 126) < [126, 136) < [136, 161) < [161, 200)]4,

Note that the output form of closeness has been changed. Now, the results are in the form of right-closed, left-open.

4. We can check the number of values in each bin by using the pd.value_counts() method:

```
pd.value_counts(category)
```

And the output is as follows:

(118, 125] 5

(135, 160] 3

(125, 135] 3

R.M.K GROUP OF INSTITUTIONS

(160, 200] 1

dtype: int64

The output shows that there are five values in the interval [118-125).

5. We can also indicate the bin names by passing a list of labels:

```
bin_names = ['Short Height', 'Average height', 'Good Height', 'Taller']
pd.cut(height, bins, labels=bin_names)
```

And the output is as follows:

[Short Height, Short Height, Short Height, Average height, Short Height, ..., Average height, Taller, Good Height, Good Height, Average height]

Length: 12

Categories (4, object): [Short Height < Average height < Good Height < Taller]

Note that we have passed at least two arguments, the data that needs to be discretized and the required number of bins. Furthermore, we have used a right=False argument to change the form of interval.

6. Now, it is essential to note that if we pass just an integer for our bins, it will compute equal-length bins based on the minimum and maximum values in the data. Let's verify what we mentioned here:

```
import numpy as np
pd.cut(np.random.rand(40), 5, precision=2)
```

In the preceding code, we have just passed 5 as the number of required bins, and the output of the preceding code is as follows:

[(0.81, 0.99], (0.094, 0.27], (0.81, 0.99], (0.45, 0.63], (0.63, 0.81], ..., (0.81, 0.99], (0.45, 0.63], (0.45, 0.63], (0.81, 0.99], (0.81, 0.99]] Length: 40

Categories (5, interval[float64]): [(0.094, 0.27] < (0.27, 0.45] < (0.45, 0.63] < (0.63, 0.81] < (0.81, 0.99]]

We can see, based on the number of bins, it created five categories.

R.M.K GROUP OF INSTITUTIONS

- A quantiles divides the range of a probability distribution into continuous intervals with alike probabilities.
- Pandas provides a qcut method that forms the bins based on sample quantiles. Let's check this with an example:

```
randomNumbers = np.random.rand(2000)
category3 = pd.qcut(randomNumbers, 4) # cut into
quartiles category3
```

And the output of the preceding code is as follows:
[(0.77, 0.999], (0.261, 0.52], (0.261, 0.52], (-0.000565, 0.261], (-0.000565, 0.261], ..., (0.77, 0.999], (0.77, 0.999], (0.261, 0.52], (-0.000565, 0.261], (0.261, 0.52]] Length: 2000 Categories (4, interval[float64]): [(-0.000565, 0.261] < (0.261, 0.52] < (0.52, 0.77] < (0.77, 0.999]]

Note that based on the number of bins, which we set to 4, it converted our data into four different categories.

- If we count the number of values in each category, we should get equal-sized bins as per our
- Let's verify that with the following comm

```
pd.value_counts(category3
```

And the output of the command is as follows:

```
(0.77, 0.999] 500
(0.52, 0.77] 500
(0.261, 0.52] 500
(-0.000565, 0.261] 500
dtype: int64
```

Our claim is hence verified. Each category contains an equal size of 500 values. Note that, similar to cut, we can also pass our own bins:
pd.qcut(randomNumbers, [0, 0.3, 0.5, 0.7, 1.0])

And the output of the preceding code is as follows: [(0.722, 0.999], (-0.000565, 0.52], (-0.000565, 0.309], (-0.000565, 0.309], (0.309, (-0.000565, 0.309], ..., (0.722, 0.999],

R.M.K GROUP OF INSTITUTIONS

(0.722, 0.999], (0.309, 0.52], (-0.000565, 0.309], (0.309, 0.52]]
Length: 2000 Categories (4, interval[float64]): [(-0.000565, 0.309] < (0.309, 0.52] < (0.52, 0.722] < (0.722, 0.999]]

Note that it created four different categories based on our code.
Here we learned how to convert continuous datasets into discrete datasets.

6. Outlier detection and filtering

- Outliers are data points that diverge from other observations for several reasons.
- During the EDA phase, one of our common tasks is to detect and filter these outliers.
- The main reason for this detection and filtering of outliers is that the presence of such outliers can cause serious issues in statistical analysis.

Lets see how to perform simple outlier detection and filtering:

1. Load the dataset that is available from the GitHub link as follows:

```
df=pd.read_csv('https://raw.githubusercontent.com/PacktPublishing
/hands-on-exploratory-data-analysis-
withpython/master/Chapter%2  04/sales.csv')
df.head(10)
```

The dataset was synthesized manually by creating a script.

R.M.K GROUP OF INSTITUTIONS

The output of the preceding df.head(10) command is shown in the following screenshot:

	Account	Company	Order	SKU	Country	Year	Quantity	UnitPrice	transactionComplete
0	123456779	Kulas Inc	99985	s9-supercomputer	Aruba	1981	5148	545	False
1	123456784	GitHub	99986	s4-supercomputer	Brazil	2001	3262	383	False
2	123456782	Kulas Inc	99990	s10-supercomputer	Montserrat	1973	9119	407	True
3	123456783	My SQ Man	99999	s1-supercomputer	El Salvador	2015	3097	615	False
4	123456787	ABC Dogma	99996	s6-supercomputer	Poland	1970	3356	91	True
5	123456778	Super Sexy Dingo	99996	s9-supercomputer	Costa Rica	2004	2474	136	True
6	123456783	ABC Dogma	99981	s11-supercomputer	Spain	2006	4081	195	False
7	123456785	ABC Dogma	99998	s9-supercomputer	Belarus	2015	6576	603	False
8	123456778	Loolo INC	99997	s8-supercomputer	Mauritius	1999	2460	36	False
9	123456775	Kulas Inc	99997	s7-supercomputer	French Guiana	2004	1831	664	True

2. Now, suppose we want to calculate the total price based on the quantity sold and the unit price. We can simply add a new column, as shown here:

```
df['TotalPrice'] = df['UnitPrice'] *
df['Quantity']
```

This should add a new column called TotalPrice, as shown in the following screenshot:

	Account	Company	Order	SKU	Country	Year	Quantity	UnitPrice	transactionComplete	TotalPrice
0	123456779	Kulas Inc	99985	s9-supercomputer	Aruba	1981	5148	545	False	2805660
1	123456784	GitHub	99986	s4-supercomputer	Brazil	2001	3262	383	False	1249346
2	123456782	Kulas Inc	99990	s10-supercomputer	Montserrat	1973	9119	407	True	3711433
3	123456783	My SQ Man	99999	s1-supercomputer	El Salvador	2015	3097	615	False	1904655
4	123456787	ABC Dogma	99996	s6-supercomputer	Poland	1970	3356	91	True	305396
5	123456778	Super Sexy Dingo	99996	s9-supercomputer	Costa Rica	2004	2474	136	True	336464
6	123456783	ABC Dogma	99981	s11-supercomputer	Spain	2006	4081	195	False	795795
7	123456785	ABC Dogma	99998	s9-supercomputer	Belarus	2015	6576	603	False	3965328
8	123456778	Loolo INC	99997	s8-supercomputer	Mauritius	1999	2460	36	False	88560
9	123456775	Kulas Inc	99997	s7-supercomputer	French Guiana	2004	1831	664	True	1215784

R.M.K GROUP OF INSTITUTIONS

Now, let's answer some questions based on the preceding table. Let's find the transaction that exceeded 3,000,000:

```
TotalTransaction = df["TotalPrice"]
TotalTransaction[np.abs(TotalTransaction) >
3000000]
```

The output of the preceding code is as follows: 2 3711433

7 3965328

13 4758900

15 5189372

17 3989325

...

9977 3475824

9984 5251134

9987 5670420

9996 3018490

Name: TotalPrice, Length: 2094, dtype: int64

Note that, in the preceding example, we have assumed that any price greater than 3,000,000 is an outlier.

Display all the columns and rows from the preceding table if TotalPrice is greater than 6741112, as follows:

```
df[np.abs(TotalTransaction) > 6741112]
```

R.M.K GROUP OF INSTITUTIONS

The output of the preceding code is the following:

	Account	Company	Order	SKU	Country	Year	Quantity	UnitPrice	transactionComplete	TotalPrice
818	123456781	Gen Power	99991	s1-supercomputer	Burkina Faso	1985	9693	696	False	6746328
1402	123456778	Will LLC	99985	s11-supercomputer	Austria	1990	9844	695	True	6841580
2242	123456770	Name IT	99997	s9-supercomputer	Myanmar	1979	9804	692	False	6784368
2876	123456772	Gen Power	99992	s10-supercomputer	Mali	2007	9935	679	False	6745865
3210	123456782	Loolo INC	99991	s8-supercomputer	Kuwait	2006	9886	692	False	6841112
3629	123456779	My SQ Man	99980	s3-supercomputer	Hong Kong	1994	9694	700	False	6785800
7674	123456781	Loolo INC	99989	s6-supercomputer	Sri Lanka	1994	9682	691	False	6828462
8645	123456789	Gen Power	99996	s11-supercomputer	Suriname	2005	9742	699	False	6809658
8684	123456785	Gen Power	99989	s2-supercomputer	Kenya	2013	9805	694	False	6804670

Note that in the output, all the TotalPrice values are greater than 6741112.
We can use any sort of conditions, either row-wise or column-wise, to detect and filter outliers.

7.7 Permutation and random sampling

Let's examine how we can perform permutation and random sampling using the pandas library:

1. With NumPy's numpy.random.permutation() function, we can rand select or permute a series of rows in a dataframe. Let's understand this with an exam

```
dat =
np.arange(80).reshape(10,8)

df = pd.DataFrame(dat)
df
```

And the output of the preceding code is as follows:

	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	8	9	10	11	12	13	14	15
2	16	17	18	19	20	21	22	23
3	24	25	26	27	28	29	30	31
4	32	33	34	35	36	37	38	39
5	40	41	42	43	44	45	46	47
6	48	49	50	51	52	53	54	55
7	56	57	58	59	60	61	62	63
8	64	65	66	67	68	69	70	71
9	72	73	74	75	76	77	78	79

R.M.K GROUP OF INSTITUTIONS

2. Next, we call the np.random.permutation() method. This method takes an argument – the length of the axis we require to be permuted – and gives an array of integers indicating the new ordering:

```
sampler =
np.random.permutation(10)  sampler
```

The output of the preceding code is as follows:

```
array([1, 5, 3, 6, 2, 4,
9, 0, 7, 8])
```

3. The preceding output array is used in ix-based indexing for the take() function from the pandas library. Check the following example for clarification:

```
df.take(sampler)
```

The output of the preceding code is as follows:

	0	1	2	3	4	5	6	7
1	8	9	10	11	12	13	14	15
5	40	41	42	43	44	45	46	47
3	24	25	26	27	28	29	30	31
6	48	49	50	51	52	53	54	55
2	16	17	18	19	20	21	22	23
4	32	33	34	35	36	37	38	39
9	72	73	74	75	76	77	78	79
0	0	1	2	3	4	5	6	7
7	56	57	58	59	60	61	62	63
8	64	65	66	67	68	69	70	71

Note that our sampler array contains array([1, 5, 3, 6, 2, 4, 9, 0, 7, 8]). Each of these array items represents the rows of the original dataframe. So, from the original dataframe, it pulls the first row, then the fifth row, then the third row, and so on. Compare this with the original dataframe output.

R.M.K GROUP OF INSTITUTIONS

1. Random sampling without replacement

To compute random sampling without replacement, follow these steps:

1. To perform random sampling without replacement, we first create a permutation array.
2. Next, we slice off the first n elements of the array where n is the desired size of the subset we want to sample.
3. Then we use the df.take() method to obtain actual samples: df.take(np.random.permutation(len(df))[:3])

The output of the preceding code is as follows:

	0	1	2	3	4	5	6	7
9	72	73	74	75	76	77	78	79
2	16	17	18	19	20	21	22	23
0	0	1	2	3	4	5	6	7

Note that in the preceding code, we only specified a sample of size 3. Hence, we only get three rows in the random sample.

7.7.2 Random sampling with replacement

To generate random sampling with replacement, follow the given steps:

1. We can generate a random sample using the numpy.random.randint() method and drawing random integers with replacement

```
sack = np.array([4, 8, -2, 7, 5])
sampler = np.random.randint(0, len(sack), size =
10)  sampler
```

We created the sampler using the np.random.randint() method. The output of the preceding code is as follows:

```
array([3, 3, 0, 4, 0, 0, 1, 2, 1, 4])
```

R.M.K GROUP OF INSTITUTIONS

2. And now, we can draw the required samples: draw =

```
sack.take(sampler)
draw
```

The output of the preceding code is as follows:

```
array([ 7,  7,  4,  5,  4, 4,  8, -2,  8,  5])
```

Compare the index of the sampler and then compare it with the original dataframe. The results are obvious in this case.

8. Computing indicators/dummy variables

Often, we need to convert a categorical variable into some dummy matrix. Especially for statistical modeling or machine learning model development, it is essential to create dummy variables.

1. Let's say we have a dataframe with data on gender and votes, as shown

```
df = pd.DataFrame({'gender': ['female', 'female', 'male', 'unknown', 'male',
'female'], 'votes': range(6, 12,
1)})
```

The output of the preceding code is as follows:

	gender	votes
0	female	6
1	female	7
2	male	8
3	unknown	9
4	male	10
5	female	11

Sometimes, however, we need to encode these values in a matrix form with 1 and 0 values.

2. We can do that using the pd.get_dummies() function: pd.get_dummies(df['gender'])

R.M.K GROUP OF INSTITUTIONS

And the output of the preceding code is as follows:

	female	male	unknown
0	1	0	0
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0
5	1	0	0

Note the pattern. There are five values in the original dataframe with three unique values of male, female, and unknown.

- Each unique value is transformed into a column and each original value into a row.
- For example, in the original dataframe, the first value is female, hence it is added as a row with 1 in the female value and the rest of them are 0 values, and so on.

3. Sometimes, we want to add a prefix to the columns. We can do that by adding the prefix argument, as shown here:

```
dummies = pd.get_dummies(df['gender'],
prefix='gender')
 dummies
```

The output of the preceding code is as follows:

	gender_female	gender_male	gender_unknown
0	1	0	0
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0
5	1	0	0

Note the gender prefix added to each of the column names.

R.M.K GROUP OF INSTITUTIONS

7.9 String manipulation

- A lot of data found online is in the form of text, and manipulating these strings is a fundamental part of data transformation.

8. Introduction to NoSQL

Few features of NoSQL databases are as follows:

1. They are open source.
2. They are non-relational.
3. They are distributed.
4. They are schema-less.
5. They are cluster friendly.
6. They are born out of 21" century web applications.

8.1 Where is it Used?

- NoSQL databases are widely used in big data and other real-time web applications.
- Refer Figure 4.1. NoSQL databases is used to stock log data which can then be pulled for analysis.



Figure 4.1 Where to use NoSQL?

- Likewise it is used to store social media data and all such data which cannot be stored and analyzed comfortably in RDBMS.

R.M.K GROUP OF INSTITUTIONS

2. What is it?

- NoSQL stands for Not Only SQL.
- These are non-relational, open source, distributed databases.
- They are hugely popular today owing to their ability to scale out or scale horizontally and the adeptness at dealing with a rich variety of data: structured, semi-structured and unstructured data.
- Refer Figure 4.2 for additional features of NoSQL.

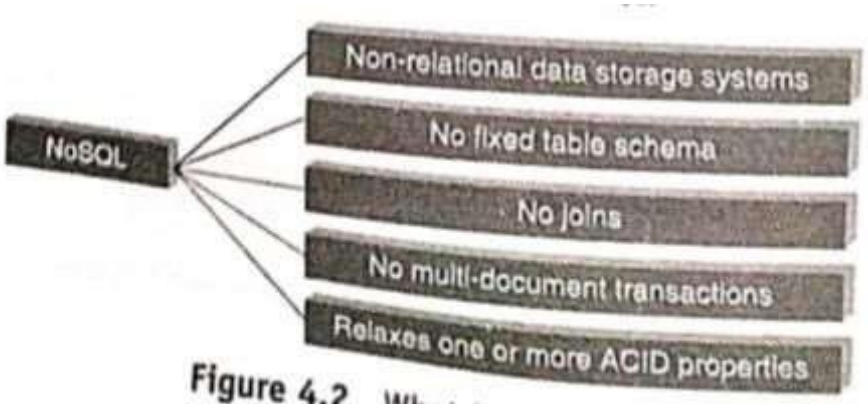


Figure 4.2 What is NoSQL?

- NoSQL databases:

1. Are non-relational: They do not adhere to relational data model. In fact, they are either key-value pairs or document-oriented or column-oriented or graph-based databases.

2. Are distributed: They are distributed meaning the data is distributed across several nodes in a cluster constituted of low-cost commodity hardware.

2.Offer no support for ACID properties (Atomicity, Consistency, Isolation, and Durability): They do not offer support for ACID properties of transactions. On the contrary, they have adherence to

Brewer's CAP (Consistency, Availability and Partition tolerance) theorem and are often seen compromising on consistency in favor of availability and partition tolerance.

4. Provide no fixed table schema: NoSQL databases are becoming increasing popular owing to their support for flexibility to the schema. They do not mandate for the data to strictly adhere to any schema structure at the time of storage.

3. Types of NoSQL Databases

NoSQL databases are non-relational. They can be broadly classified into the following:

1. Key-value or the big hash table.
2. Schema-less.

1. **Key-value:** It maintains a big hash table of keys and values. For exam Dynamo, Redis, Riak,

Sample Key-Value Pair in Key-Value Database

Key	Value
First Name	Simmonds
Last Name	David

2. **Document:** It maintains data in collections constituted of documents. For example, MongoDB, Apache CouchDB, Couchbase, MarkLogic, etc.

Sample Document in Document Database

```
{
"Book Name":    "Fundamentals of Business Analytics",
"Publisher":    "Wiley India",
"Year of Publication":    "2011"
}
```

R.M.K GROUP OF INSTITUTIONS

3. Column: Each storage block has data from only one column. For example: Cassandra, HBase, etc.

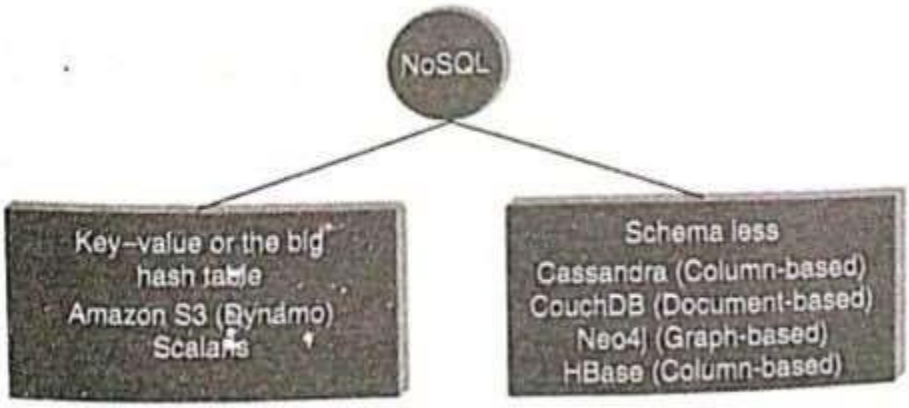
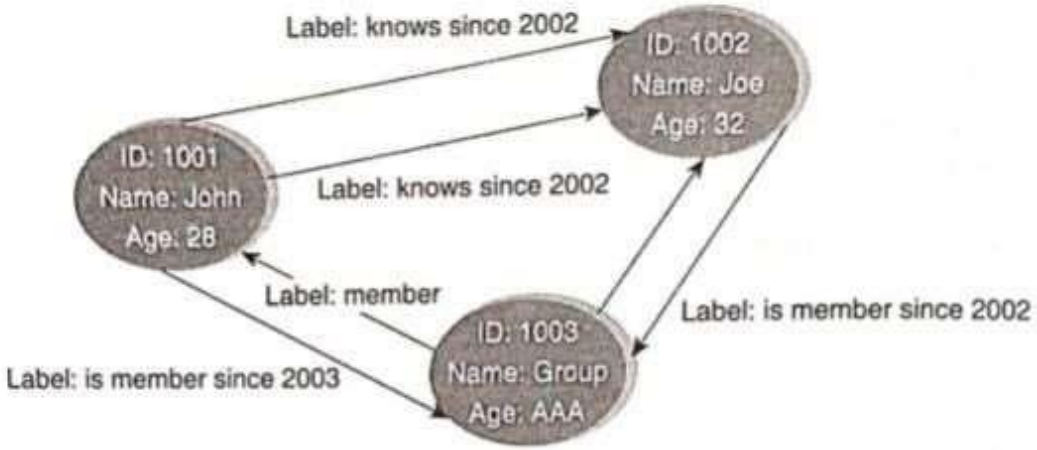


Figure 4.3 Types of NoSQL databases.

4. Graph: They are also called network database. A graph stores data in nodes. For example. Neo4j, HyperGraphDB, etc.

Sample Graph in Graph Database



R.M.K GROUP OF INSTITUTIONS

Table 4.1 Popular schema-less databases

Key–Value Data Store	Column-Oriented Data Store	Document Data Store	Graph Data Store
• Riak	• Cassandra	• MongoDB	• InfiniteGraph
• Redis	• HBase	• CouchDB	• Neo4j
• Membase	• HyperTable	• RavenDB	• AllegroGraph

8.4 Advantages of NoSQL

1. Can easily scale up and down: NoSQL database supports scaling rapidly and elastically and even allows to scale to the cloud.

(a) Cluster scale: It allows distribution of database across 100+ nodes often in multiple data centers.

(b) Performance scale: It sustains over 100,000+ database reads and writes per second.

(c) Data scale: It supports housing of 1 billion+ documents in the database.

2. Doesn't require a pre-defined schema: NoSQL does not require any adherence to pre-defined schema. It is pretty flexible. For example, if we look at MongoDB, the documents (equivalent of records in RDBMS) in a collection (equivalent of table in RDBMS) can have different sets of key-value pairs.

{_id: 101,"BookName": "Fundamentals of Business Analytics", "AuthorName": "Seema Acharya", "Publisher": "Wiley India"}
{_id:102, "BookName":"Big Data and Analytics"}

3. Cheap, easy to implement: Deploying NoSQL properly allows for all of the benefits of scale, high availability, fault tolerance, etc. while also lowering operational costs.

R.M.K GROUP OF INSTITUTIONS

4. Relaxes the data consistency requirement: NoSQL databases have adherence to CAP theorem (Consistency, Availability, and Partition tolerance). Most of the NoSQL databases compromise on consistency in favor of availability and partition tolerance. However, they do go for eventual consistency.

5. Data can be replicated to multiple nodes and can be partitioned: There are two terms here:

(a) **Sharding:** Sharding is when different pieces of data are distributed across multiple servers.

- NoSQL databases support auto-sharding; this means that they can natively and automatically spread data across an arbitrary number of servers, without requiring the application to even be aware of the composition of the server pool.
- Servers can be added or removed from the data layer without application downtime. This would mean that data and query load are automatically balanced across servers, and when a server goes down, it can be quickly and transparently replaced with no application disruption.

(b) **Replication:** Replication is when multiple copies of data are stored across the cluster and even across data centers. This promises high availability and fault tolerance.

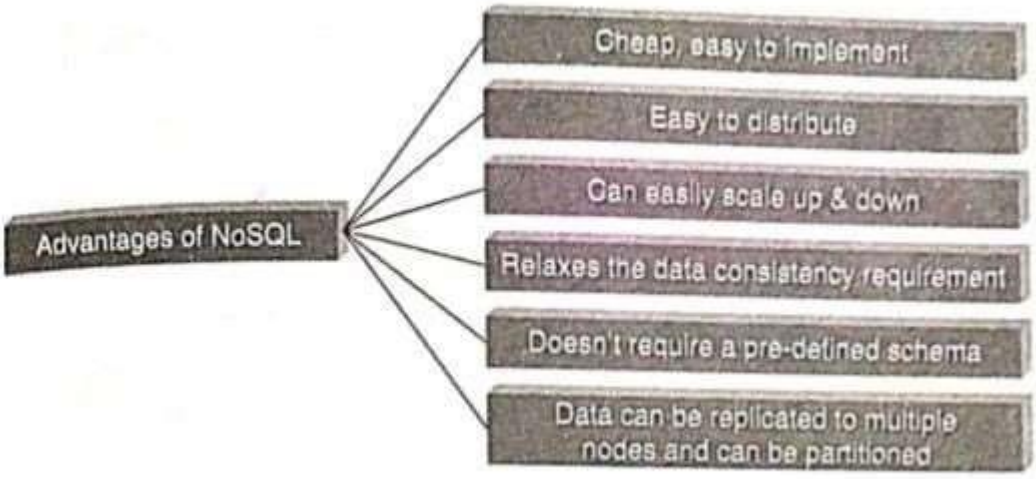


Figure 4.4 Advantages of NoSQL.

8.5 Use of NoSQL in Industry

NoSQL is being put to use in varied industries. They are used to support analysis for applications such as web user data analysis, log analysis, sensor feed analysis, making recommendations for upsell and cross-sell etc. Refer Figure 4.6.

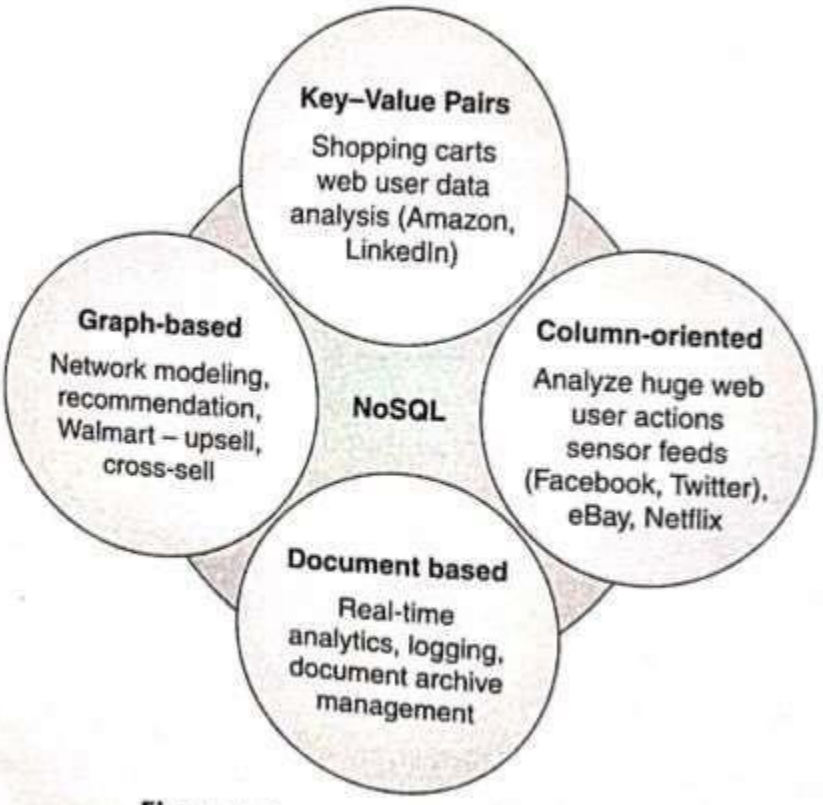


Figure 4.6 Use of NoSQL in industry.

8.6 NoSQL Vendors

Few popular NoSQL vendors are:

Table 4.2 Few popular NoSQL vendors

Company	Product	Most Widely Used by
Amazon	DynamoDB	LinkedIn, Mozilla
Facebook	Cassandra	Netflix, Twitter, eBay
Google	BigTable	Adobe Photoshop

8.7 SQL versus NoSQL

Few salient differences between SQL and NoSQL.

Table 4.3 SQL versus NoSQL

SQL	NoSQL
Relational database	Non-relational, distributed database
Relational model	Model-less approach
Pre-defined schema	Dynamic schema for unstructured data
Table based databases	Document-based or graph-based or wide column store or key-value pairs databases
Vertically scalable (by increasing system resources)	Horizontally scalable (by creating a cluster of commodity machines)
Uses SQL	Uses UnQL (Unstructured Query Language)
Not preferred for large datasets	Largely preferred for large datasets
Not a best fit for hierarchical data	Best fit for hierarchical storage as it follows the key-value pair of storing data similar to JSON (Java Script Object Notation)
Emphasis on ACID properties	Follows Brewer's CAP theorem
Excellent support from vendors	Relies heavily on community support
Supports complex querying and data keeping needs	Does not have good support for complex querying
Can be configured for strong consistency	Few support strong consistency (e.g., MongoDB), some others can be configured for eventual consistency (e.g., Cassandra)
Examples: Oracle, DB2, MySQL, MS SQL, PostgreSQL, etc.	Examples: MongoDB, HBase, Cassandra, Redis, Neo4j, CouchDB, Couchbase, Riak, etc.

8.8 NewSQL

We need a database that has the same scalable performance of NoSQL systems for On Line Transaction Processing (OLTP) while still maintaining the ACID guarantees of a traditional database. This new modern RDBMS is called NewSQL. It supports relational data model and uses SQL as their primary interface.

8.8.1 Characteristics of NewSQL

NewSQL is based on the shared nothing architecture with a SQL interface for application interaction.

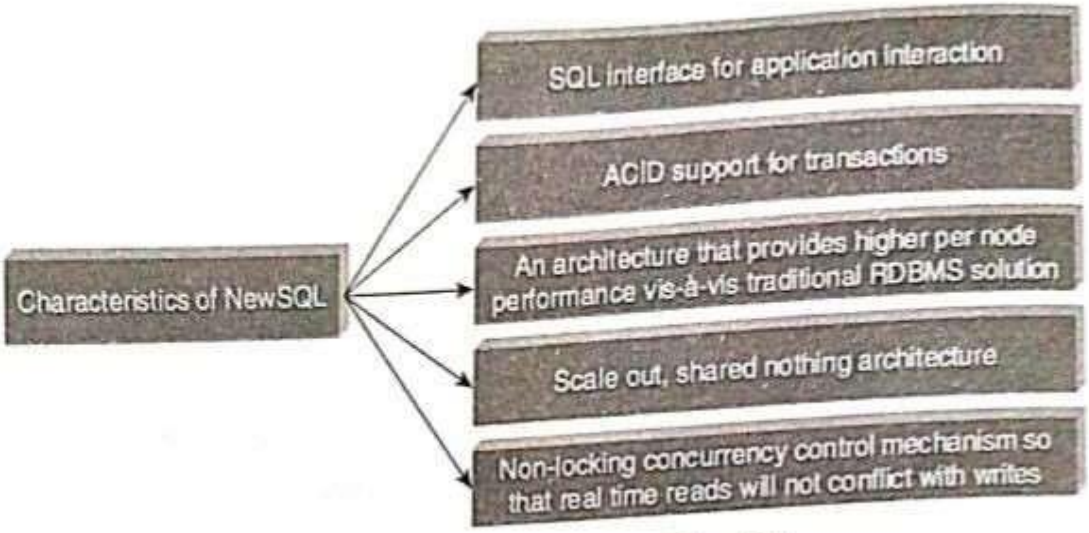


Figure 4.7 Characteristics of NewSQL.

8.9 Comparison of SQL, NoSQL and NewSQL

Table 4.4 Comparative study of SQL, NoSQL and NewSQL

	SQL	NoSQL	NewSQL
Adherence to ACID properties	Yes	No	Yes
OLTP/OLAP	Yes	No	Yes
Schema rigidity	Yes	No	Maybe
Adherence to data model	Adherence to relational model		
Data Format Flexibility	No	Yes	Maybe
Scalability	Scale up Vertical Scaling	Scale out Horizontal Scaling	Scale out
Distributed Computing	Yes	Yes	Yes
Community Support	Huge	Growing	Slowly growing

R.M.K GROUP OF INSTITUTIONS

9. WHAT IS MONGODB?

MongoDB is

1. Cross-platform.
2. Open source.
3. Non-relational.
4. Distributed.
5. NoSQL.
6. Document-oriented data store

1. WHY MONGODB?

- Few of the major challenges with traditional RDBMS are dealing with large volumes of data, rich variety of data- particularly unstructured data and meeting up to the scale needs of enterprise data.
- The need is for a database that can scale out or scale horizontally to meet the scale requirements, has flexibility with respect to schema, is fault tolerant, is consistent and partition tolerant, and can be easily distributed over a multitude of nodes in a cluster.

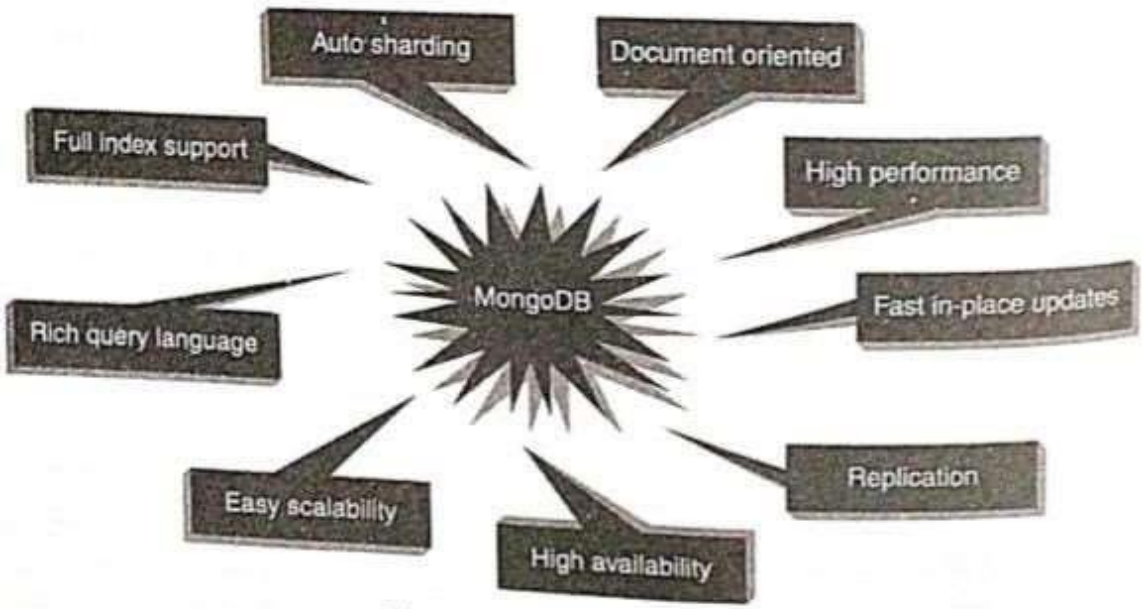


Figure 6.1 Why MongoDB?

9.2 TERMS USED IN RDBMS AND MONGODB

RDBMS	MongoDB
Database	Database
Table	Collection
Record	Document
Columns	Fields / Key Value pairs
Index	Index
Joins	Embedded documents
Primary Key	Primary key (_id is a identifier)

	MySQL	Oracle	MongoDB
Database Server	MySqld	Oracle	Mongod
Database Client	MySql	SQL Plus	mongo

9.2.1 Create Database

The syntax for creating database is as follows use

To create a database by the name "myDB" the syntax is use myDB

```
> use myDB;
switched to db myDB
>
```

To confirm the existence of our database, type the command at the MongoDB shell:

db

```
> db;
myDB
>
```

To get a list of all databases, type the below command: show dbs

```
> show dbs;
admin   (empty)
local   0.078GB
test    0.078GB
>
```

Notice that the newly created database, "myDB" does not show up in the list above. The reason is that the database needs to have at least one document to show up in the list.

The default database in MongoDB is test. If one does not create any database, all collections are by default stored in the test database.

9.2.2 Drop Database

The syntax to drop database is as follows:

```
db.dropDatabase();
```

To drop the database, "myDB", first ensure that you are currently placed in "myDB" database and then use the db.dropDatabase() command to drop the database.

```
use
db.dropDatabase();
```

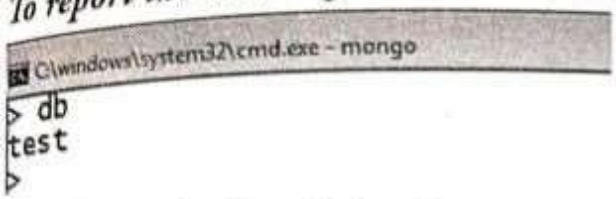
Confirm if the database "myDB" has been dropped.

```
> db.dropDatabase();
{ "dropped" : "myDB", "ok" : 1 }
>
```

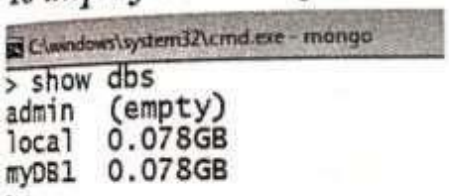
If no database is selected, the default database "test" is dropped.

R.M.K GROUP OF INSTITUTIONS

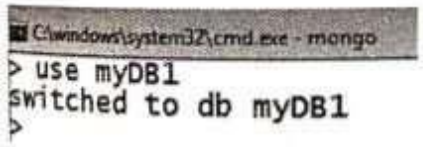
To report the name of the current database:



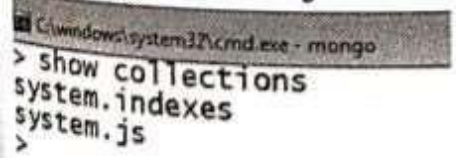
To display the list of databases:



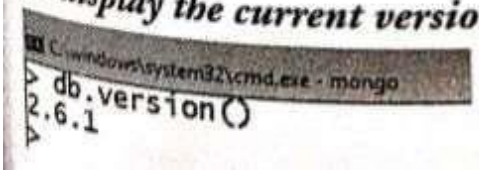
To switch to a new database, for example, myDB1:



To display the list of collections (tables) in the current database:



To display the current version of the MongoDB server:



R.M.K GROUP OF INSTITUTIONS

To display the statistics that reflect the use state of a database:

```
C:\windows\system32\cmd.exe - mongo
> db.stats()
{
        "db" : "myDB1",
        "collections" : 3,
        "objects" : 6,
        "avgObjSize" : 122.66666666666667,
        "dataSize" : 736,
        "storageSize" : 24576,
        "numExtents" : 3,
        "indexes" : 1,
        "indexSize" : 8176,
        "fileSize" : 67108864,
        "nsSizeMB" : 16,
        "dataFileVersion" : {
                "major" : 4,
                "minor" : 5
        },
        "extentFreeList" : {
                "num" : 14,
                "totalSize" : 974848
        },
        "ok" : 1
}
>
```

Type in db.help() in the MongoDB client to get a list of commands:

```
C:\windows\system32\cmd.exe - mongo
> db.help();
DB methods:
        db.adminCommand(nameOrDocument) - switches to 'admin' db, and runs comma
nd [ just calls db.runCommand(...) ]
        db.auth(username, password)
        db.cloneDatabase(fromhost)
        db.commandHelp(name) returns the help for the command
        db.copyDatabase(fromdb, todb, fromhost)
        db.createCollection(name, { size : ..., capped : ..., max : ... } )
        db.createUser(userDocument)
        db.currentOp() displays currently executing operations in the db
        db.dropDatabase()
        db.eval(func, args) run code server-side
        db.fsyncLock() flush data to disk and lock server for backups
        db.fsyncUnlock() unlocks server following a db.fsyncLock()
        db.getCollection(cname) same as db['cname'] or db.cname
```

9.3 RDBMS vs MongoDB

	RDBMS	MongoDB
Insert	Insert into Students (StudRollNo, StudName, Grade, Hobbies, DOJ) Values ('S101', 'Simon David', 'VII', 'Net Surfing', '10-Oct-2012')	db.Students.insert({_id:1, StudRollNo: 'S101', StudName: 'Simon David', Grade: 'VII', Hobbies: 'Net Surfing', DOJ: '10-Oct-2012'});
Update	Update Students set Hobbies ='Ice Hockey' where StudRollNo ='S101'	db.Students.update({StudRollNo: 'S101'}, {$set: {Hobbies : 'Ice Hockey'}})
	Update Students Set Hobbies ='Ice Hockey'	db.Students.update({},{$set: {Hobbies: 'Ice Hockey' }}, {multi:true})
Delete	Delete from Students where StudRollNo = 'S101'	db.Students.remove ({StudRollNo : 'S101'})
	Delete From Students	db.Students.remove({})
Select	Select * from Students	db.Students.find() db.Students.find().pretty()
	Select * from students where StudRollNo = 'S101'	db.Students.find({StudRollNo: 'S101'})
	Select StudRollNo, StudName, Hobbies from Students	db.Students.find({},{StudRollNo:1, StudName:1, Hobbies:1, _id:0})
	Select StudRollNo, StudName, Hobbies from Students where StudRollNo = 'S101'	db.Students.find({StudRollNo: 'S101'}, {StudRollNo : 1, StudName: 1, Hobbies : 1, _id:0})
	Select StudRollNo, StudName, Hobbies From Students Where Grade ='VII' and Hobbies ='Ice Hockey'	db.Students.find({Grade: 'VII' , Hobbies: 'Ice Hockey'}, {StudRollNo : 1, StudName: 1, Hobbies : 1, _id:0})

Select StudRollNo, StudName, Hobbies From Students Where Grade ='VII' or Hobbies = 'Ice Hockey'	db.Students.find({ $or: [{Grade: 'VII' , Hobbies: 'Ice Hockey'}], StudRollNo : 1, StudName: 1, Hobbies : 1, _id:0})
Select * From Students Where StudName like 'S%'	db.Students.find({ StudName: /^S/}).pretty()

9.4 MONGODB QUERY LANGUAGE

CRUD (Create, Read Update, and Delete) operations in MongoDB

Create -> Creation of data is done using insert() or update() or save() method.

Read -> Reading the data is performed using the find() method.

Update -> Update to data is accomplished using the update() method with UPSERT set to false

Delete -> a document is Deleted using the remove() method.

The organization is as

Objective: What is it that we are trying to achieve

Input: What is the input that has been given to us to act

Act: The actual statement/command to accomplish the task at

Outcome: The result/output as a consequence of executing the statements

Alternatives we have also provided the equivalent in RDBMS such as Oracle.

Objective: To create a collection by the name "Person". Let us take a look at the collection list prior to the creation of the new collection "Person":

```
> show collections
Students
food
system.indexes
system.js
>
```

Act: The statement to create the collection is

db.createCollection("Person")

R.M.K GROUP OF INSTITUTIONS

```
C:\windows\system32\cmd.exe - mongo
> db.createCollection("Person");
{ "ok" : 1 }
>
```

Outcome: Below is the collection list after the creation of the new collection "Person":

```
C:\windows\system32\cmd.exe - mongo
> show collections;
Person
Students
food
system.indexes
system.js
>
```

Objective: To drop a collection by the name "food".

Take a look at the current collection list:

```
C:\windows\system32\cmd.exe - mongo
> show collections;
Person
Students
food
system.indexes
system.js
>
```

Act: The statement to drop the collection is

```
db.food.drop();
```

```
C:\windows\system32\cmd.exe - mongo
> db.food.drop();
true
>
```

Outcome: The collection list after the execution of the statement is as follows:

```
C:\windows\system32\cmd.exe - mongo
> show collections
Person
Students
system.indexes
system.js
>
```

6.5.1 Insert Method

We now explain the syntax of insert method.

```
db.students.insert(          <-------- Collection
{
RollNo: 101,                 <-------- Field: value
Age: 19,                     <-------- Field: value
ContactNo:0123456789,        <-------- Field: value
EmailID:Sample@abc.com       <-------- Field: value
}
)
```

Objective: To create a collection by the name "Students" and insert documents.

Input: Check the list of existing collections.

```
C:\windows\system32\cmd.exe - mongo
> show collections
system.indexes
system.js
>
```

Act: Create a collection by the name "Students" and store the following data in it.

db.Students.insert({_id:1, StudName:"Michelle Jacintha", Grade: "VII", Hobbies: "Internet Surfing"});

```
> db.Students.insert({_id:1, StudName:"Michelle Jacintha", Grade: "VII", Hobbies: "Internet Surfing"});
WriteResult({ "nInserted" : 1 })
>
```

Outcome: Check if the collection has been successfully created.

```
C:\windows\system32\cmd.exe - mongo
> show collections
Students
system.indexes
system.js
>
```

Check if the document for Student "Michelle Jacintha" has been successfully inserted into the "Students" collection.

```
> db.Students.find();
{ "_id" : 1, "StudName" : "Michelle Jacintha", "Grade" : "VII", "Hobbies" : "Internet Surfing" }
>
```

R.M.K GROUP OF INSTITUTIONS

To format the result, one can add the pretty() method to the operation.

```
> db.Students.find().pretty();
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
>
```

Objective: Insert another document into the collection.

Input: Check the documents in the "Students" collection before proceeding.

```
> db.Students.find().pretty();
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
>
```

Act:

```
db.Students.insert({_id:2, StudName:"Mabel Mathews", Grade: "VII", Hobbies: "Baseball"});
```

```
> db.Students.insert({_id:2, StudName:"Mabel Mathews", Grade: "VII", Hobbies: "Baseball"});
WriteResult({ "nInserted" : 1 })
>
```

Outcome:

```
> db.Students.find().pretty();
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
{
        "_id" : 2,
        "StudName" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
>
```

R.M.K GROUP OF INSTITUTIONS

6.5.2 Save() Method

We now explain the save() method. The save() method will insert a new document if the document with the specified _id does not exist. However, if a document with the specified id exists, it replaces the existing document with the new one.

Objective: Insert the document of "Hersch Gibbs" into the Students collection using the Update method. First try with upsert set to false and then with upsert set to true.

Input: Check the documents in the "Students" collection before proceeding.

```
> db.Students.find();
{ "_id" : 1, "StudName" : "Michelle Jacintha", "Grade" : "VII", "Hobbies" : "Internet Surfing" }
{ "_id" : 2, "StudName" : "Mabel Mathews", "Grade" : "VII", "Hobbies" : "Baseball" }
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
{ "_id" : ObjectId("546dd0e0a7fba710799bb94d"), "StudName" : "Vamsi Bapat", "Grade" : "VI" }
```

Act: *Update method with upsert set to false.*

db.Students.update({_id:4, StudName:"Hersch Gibbs", Grade: "VII"},{$set:{Hobbies: "Graffiti"}}, {upsert:false});

```
> db.Students.update({_id:4, StudName:"Hersch Gibbs", Grade: "VII"},{$set:{Hobbies: "Graffiti"}}, {upsert:false});
WriteResult({ "nMatched" : 0, "nUpserted" : 0, "nModified" : 0 })
>
```

As evident from the above display (nUpserted : 0), no document has been inserted because upsert is set to false.

Update method with upsert set to true.

db.Students.update({_id:4, StudName:"Hersch Gibbs", Grade: "VII"},{$set:{Hobbies: "Graffiti"}}, {upsert:true});

```
> db.Students.update({_id:4, StudName:"Hersch Gibbs", Grade: "VII"},{$set:{Hobbies: "Graffiti"}}, {upsert:true});
WriteResult({ "nMatched" : 0, "nUpserted" : 1, "nModified" : 0, "_id" : 4 })
>
```

nUpserted: 1 implies that a document with _id:4 has been inserted.

Outcome: Confirm the presence of the document of "Hersch Gibbs" in the "Students" collection.

```
> db.Students.find();
{ "_id" : 1, "StudName" : "Michelle Jacintha", "Grade" : "VII", "Hobbies" : "Internet Surfing" }
{ "_id" : 2, "StudName" : "Mabel Mathews", "Grade" : "VII", "Hobbies" : "Baseball" }
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
{ "_id" : ObjectId("546dd0e0a7fba710799bb94d"), "StudName" : "Vamsi Bapat", "Grade" : "VI" }
{ "_id" : 4, "Grade" : "VII", "StudName" : "Hersch Gibbs", "Hobbies" : "Graffiti" }
>
```

R.M.K GROUP OF INSTITUTIONS

6.5.3 Adding a New Field to an Existing Document – Update Method

We now discuss the syntax of update method.

```
db.students.update(          <-------  Collection
{Age: {$gt 18}},             <-------  Update Criteria
{$set: {Status: "A" }}       <-------  Update Action
{multi:true}                 <-------  Update Option
)
```

Objective: To add a new field "Location" with value "Newark" to the document (_id:4) of "Students" collection.

Input: Check the document (_id:4) in the "Students" collection before proceeding.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({_id:4}).pretty();
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
>
```

Act:

```
db.Students.update({_id:4},{$set:{Location:"Newark"}});
```

```
C:\windows\system32\cmd.exe - mongo
> db.Students.update({_id:4},{$set:{Location:"Newark"}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
```

Outcome: Confirm that the new field "Location" with value "Newark" has been added to document (_id:4) in the "Students" collection.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({_id:4}).pretty();
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti",
        "Location" : "Newark"
}
>
```

R.M.K GROUP OF INSTITUTIONS

6.5.4 Removing an Existing Field from an Existing Document – Remove Method

In this section we will explain the syntax of remove method.

```
db.students.remove(          <-------- Collection
{Age: {$gt 18}},             <-------- Remove Criteria
)
```

Objective: To remove the field "Location" with value "Newark" in the document (_id:4) of "Students" collection.

Input: Check the document (_id:4) in the "Students" collection before proceeding.

```
> db.Students.find({_id:4}).pretty();
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti",
        "Location" : "Newark"
}
>
```

Act:

```
db.Students.update({_id:4},{$unset:{Location:"Newark"}});
```

```
> db.Students.update({_id:4},{$unset:{Location:"Newark"}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
```

Outcome: Confirm if the stated field ("Location") has been dropped from the document (_id:4) of the "Students" collection.

```
> db.Students.find({_id:4}).pretty();
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
>
```

6.5.5 Finding Documents based on Search Criteria – Find Method

The syntax of find method is as follows:

```
db.students.find(            <-------- Collection
{Age: {$gt 18}},             <-------- Selection Criteria
{RollNo:1,Age:1,_id:1}       <-------- Projection
).limit(10)                  <-------- Cursor Modifier
```

Objective: To search for documents from the "Students" collection based on certain search criteria.

Input: Check the documents in the "Students" collection before proceeding.

R.M.K GROUP OF INSTITUTIONS

```
> db.Students.find({}).pretty();
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"

        "_id" : 3,
        "Grade" : "VII",
        "StudName" : "Aryan David",
        "Hobbies" : "Chess"

        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"

        "_id" : ObjectId("5464849889ad1ab07d489b7f"),
        "StudName" : "Vamsi Bapat",
        "Grade" : "VI"

        "_id" : 2,
        "StudName" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
```

Act: Find the document wherein the "StudName" has value "Aryan David".

db.Students.find({StudName:"Aryan David"});

Outcome:

```
> db.Students.find({StudName:"Aryan David"});
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
>
```

To format the above output, use the pretty() method:

db.Students.find({StudName:"Aryan David"}).pretty();

```
> db.Students.find({StudName:"Aryan David"}).pretty();
{
        "_id" : 3,
        "Grade" : "VII",
        "StudName" : "Aryan David",
        "Hobbies" : "Chess"
}
```

RDBMS equivalent:

Select *

From Students

Where StudName like 'Aryan David';

```
SQL> select * from Students where StudName like 'Aryan David';
STUDR STUDNAME                    GRADE HOBBIES
----- --------------------------- ----- ------------------------------
3     Aryan David                 VII   Chess
SQL>
```

R.M.K GROUP OF INSTITUTIONS

Objective: To find documents from the Students collection where the StudName begins with "M".

Act:

```
db.Students.find({StudName:{$regex:"^M"}}).pretty();
```

Outcome:

```
> db.Students.find({StudName:{$regex:"^M"}}).pretty();
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
{
        "_id" : 2,
        "StudName" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
>
```

RDBMS Equivalent:

```
Select *
        From Students
                Where StudName like 'M%';
```

```
SQL> select * from Students where StudName like 'M%';

STUDR STUDNAME                 GRADE HOBBIES
----- ------------------------ ----- ------------------------------
1     Michelle Jacintha        VII   Internet surfing
2     Mabel Mathews            VII   Baseball

SQL>
```

6.5.6 Dealing with NULL Values

Objective: To add a new field with null value in existing documents (_id:3 and _id:4) of Students collection. A NULL is a missing or unknown value. When we place NULL as a value for a field, it implies that currently we do not know the value or the value is missing. We can always update the value of the field once we know it.

Input: Before we execute the commands to update documents with a null value in a column, let us first view the two documents.

```
db.Students.find({$or:[{_id:3},{_id:4}]})
```

```
> db.Students.find({$or:[{_id:3},{_id:4}]});
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
{ "_id" : 4, "Grade" : "VII", "StudName" : "Hersch Gibbs", "Hobbies" : "Graffiti" }
>
```

Act: Update the documents with NULL values in the "Location" column.

```
db.Students.update({_id:3},{$set:{Location:null}});
db.Students.update({_id:4},{$set:{Location:null}});
```

```
> db.Students.update({_id:3},{$set:{Location:null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
> db.Students.update({_id:4},{$set:{Location:null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
```

R.M.K GROUP OF INSTITUTIONS

RDBMS Equivalent:

```
Update Students
        Set Location = null
                Where StudRollNo in ('3','4');
```

```
SQL> update Students set Location = null where StudRollNo in ('3','4');
2 rows updated.
SQL>
```

Outcome: To search for NULL values in Location column.

```
db.Students.find({Location:{$eq:null}});
```

```
> db.Students.find({Location:{$eq:null}});
{ "_id" : 1, "StudName" : "Michelle Jacintha", "Grade" : "VII", "Hobbies" : "Internet Surfing" }
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess", "Location" : null }
{ "_id" : 4, "Grade" : "VII", "StudName" : "Hersch Gibbs", "Hobbies" : "Graffiti", "Location" : null }
{ "_id" : ObjectId("5464849889ad1ab07d489b7f"), "StudName" : "Vamsi Bapat", "Grade" : "VI" }
{ "_id" : 2, "StudName" : "Mabel Mathews", "Grade" : "VII", "Hobbies" : "Baseball" }
>
```

The above statement displays documents which either have NULL values in the location column or do not have the location column at all.

RDBMS Equivalent:

```
Select *
        From Students
                Where Location is Null;
```

```
SQL> select * from Students where Location is NULL;

STUDR STUDNAME                 GRADE HOBBIES                        LOCATION
----- ------------------------ ----- ------------------------------ ----------
1     Michelle Jacintha        VII   Internet surfing
3     Aryan David              VII   Chess
4     Mabel Mathews            VII   Baseball
      Hersch Gibbs             VII   Graffiti
      Vamsi Bapat              VI

SQL>
```

R.M.K GROUP OF INSTITUTIONS

6.5.7 Count, Limit, Sort, and Skip

Objective: To find the number of documents in the Students collection.

Act:

```
db.Students.count()
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.Students.count()
5
>
```

Objective: To find the number of documents in the Students collection wherein the Grade is VII.

Act:

```
db.Students.count({Grade:"VII"});
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.Students.count({Grade:"VII"});
4
>
```

Objective: To retrieve the first 3 documents from the Students collection wherein the Grade is VII.

Act:

```
db.Students.find({Grade:"VII"}).limit(3).pretty();
```

Outcome:

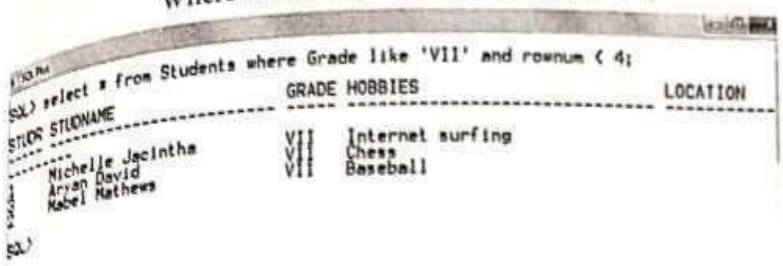
```
> db.Students.find({Grade:"VII"}).limit(3).pretty();
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
{
        "_id" : 3,
        "Grade" : "VII",
        "StudName" : "Aryan David",
        "Hobbies" : "Chess"
}
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
>
```

RDBMS Equivalent:

Select *

R.M.K GROUP OF INSTITUTIONS

From Students
Where Grade like 'VII' and rownum < 4;



Objective: To sort the documents from the Students collection in the ascending order of StudName.

Act:

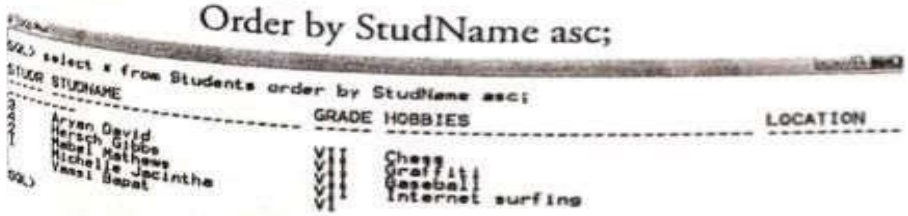
```
db.Students.find().sort({StudName:1}).pretty();
```

Outcome:

```
> db.Students.find().sort({StudName:1}).pretty();
{
    "_id" : 3,
    "Grade" : "VII",
    "StudName" : "Aryan David",
    "Hobbies" : "Chess"
}
{
    "_id" : 4,
    "Grade" : "VII",
    "StudName" : "Hersch Gibbs",
    "Hobbies" : "Graffiti"
}
{
    "_id" : 2,
    "StudName" : "Mabel Mathews",
    "Grade" : "VII",
    "Hobbies" : "Baseball"
}
{
    "_id" : 1,
    "StudName" : "Michelle Jacintha",
    "Grade" : "VII",
    "Hobbies" : "Internet Surfing"
}
{
    "_id" : ObjectId("5464849889ad1ab07d489b7f"),
    "StudName" : "Vamsi Bapat",
    "Grade" : "VI"
}
```

RDBMS Equivalent:

Select *
From Students
Order by StudName asc;



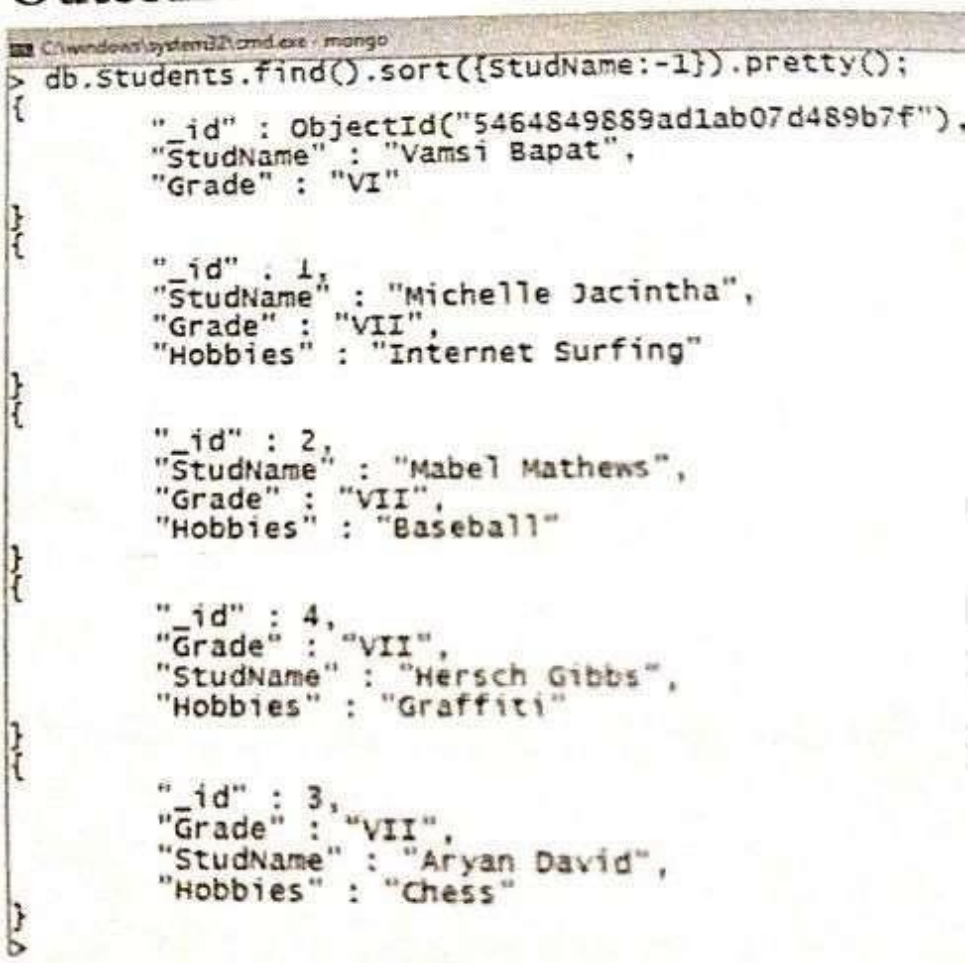
R.M.K GROUP OF INSTITUTIONS

Objective: To sort the documents from the Students collection in the descending order of StudName.

Act:

```
db.Students.find().sort({StudName:-1}).pretty();
```

Outcome:



```
C:\windows\system32\cmd.exe - mongo
> db.Students.find().sort({StudName:-1}).pretty();
{
        "_id" : ObjectId("5464849889ad1ab07d489b7f"),
        "StudName" : "Vamsi Bapat",
        "Grade" : "VI"
}
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
{
        "_id" : 2,
        "StudName" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
{
        "_id" : 3,
        "Grade" : "VII",
        "StudName" : "Aryan David",
        "Hobbies" : "Chess"
}
```

Objective: To sort the documents from the Students collection first on Grade in ascending order and then on Hobbies in descending order.

Act:

```
db.Students.find().sort({Grade:1, Hobbies:-1}).pretty();
```

Outcome:



```
> db.Students.find().sort({Grade:1, Hobbies:-1}).pretty();
{
        "_id" : ObjectId("5464849889ad1ab07d489b7f"),
        "Studname" : "Vamsi Bapat",
        "Grade" : "VI"
}
{
        "_id" : 1,
        "Studname" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
{
        "_id" : 4,
        "Grade" : "VII",
        "Studname" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
{
        "_id" : 3,
        "Grade" : "VII",
        "Studname" : "Aryan David",
        "Hobbies" : "Chess"
}
{
        "_id" : 2,
        "Studname" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
```

Objective: To sort the documents from the Students collection first on Grade in ascending order and then on Hobbies in ascending order.

Act:

```
db.Students.find().sort({Grade:1, Hobbies:1}).pretty();
```

Outcome:



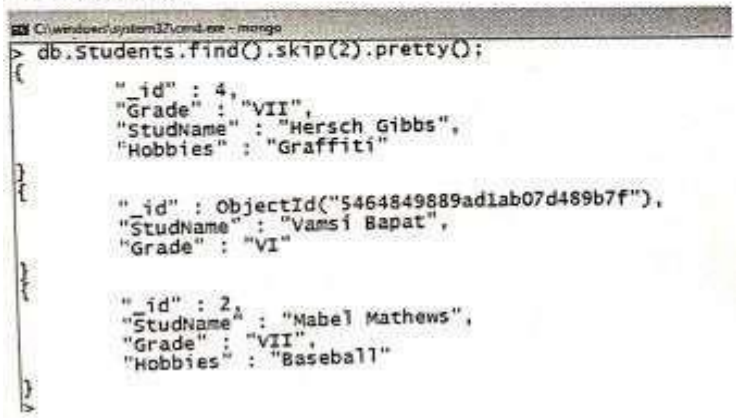
```
> db.Students.find().sort({Grade:1, Hobbies:1}).pretty();
{
        "_id" : ObjectId("5464849889ad1ab07d489b7f"),
        "Studname" : "Vamsi Bapat",
        "Grade" : "VI"
}
{
        "_id" : 2,
        "Studname" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
{
        "_id" : 3,
        "Grade" : "VII",
        "Studname" : "Aryan David",
        "Hobbies" : "Chess"
}
{
        "_id" : 4,
        "Grade" : "VII",
        "Studname" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
{
        "_id" : 1,
        "Studname" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
```

Objective: To skip the first 2 documents from the Students collection.

Act:

```
db.Students.find().skip(2).pretty();
```

Outcome:

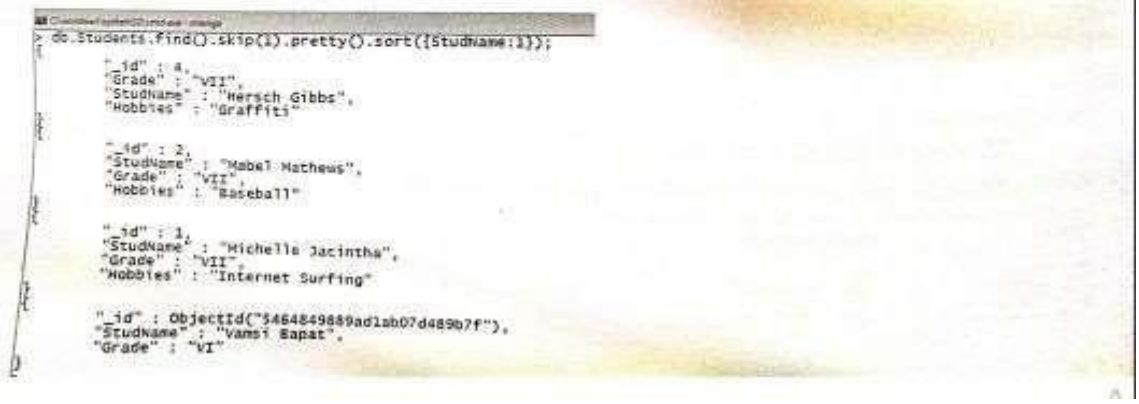


Objective: To sort the documents from the Students collection and skip the first document from the output.

Act:

```
db.Students.find().skip(1).pretty().sort({StudName:1});
```

Outcome:



Objective: To display the last 2 records from the Students collection.

Act:

```
db.Students.find().pretty().skip(db.Students.count()-2);
```

Outcome:

```
> db.Students.find().pretty().skip(db.Students.count()-2);
{
    "_id" : ObjectId("5464849889ad1ab07d489b7f"),
    "StudName" : "Vamsi Bapat",
    "Grade" : "VI"
}
{
    "_id" : 2,
    "StudName" : "Mabel Mathews",
    "Grade" : "VII",
    "Hobbies" : "Baseball"
}
```

Objective: To retrieve the third, fourth, and fifth document from the Students collection.

Act:

```
db.Students.find().pretty().skip(2).limit(3);
```

Outcome:

```
> db.Students.find().pretty().skip(2).limit(3);
{
    "_id" : 4,
    "Grade" : "VII",
    "StudName" : "Hersch Gibbs",
    "Hobbies" : "Graffiti"
}
{
    "_id" : ObjectId("5464849889ad1ab07d489b7f"),
    "StudName" : "Vamsi Bapat",
    "Grade" : "VI"
}
{
    "_id" : 2,
    "StudName" : "Mabel Mathews",
    "Grade" : "VII",
    "Hobbies" : "Baseball"
}
```

6.5.8 Arrays

Objective: To create a collection by the name "food" and then insert documents into the "food" collection. Each document should have a "fruits" array.

Act:

```
db.food.insert({_id:1,fruits:[ 'banana','apple','cherry' ] })
db.food.insert({_id:2,fruits:[ 'orange','butterfruit','mango' ]})
db.food.insert({_id:3,fruits:[ 'pineapple','strawberry','grapes']});
db.food.insert({_id:4,fruits:[ 'banana','strawberry','grapes']});
db.food.insert({_id:5,fruits:[ 'orange','grapes']});
```

```
> db.food.insert({_id:1,fruits:[ 'banana','apple','cherry' ] })
WriteResult({ "nInserted" : 1 })
> db.food.insert({_id:2,fruits:[ 'orange','butterfruit','mango' ]})
WriteResult({ "nInserted" : 1 })
> db.food.insert({_id:3,fruits:[ 'pineapple','strawberry','grapes']});
WriteResult({ "nInserted" : 1 })
> db.food.insert({_id:4,fruits:[ 'banana','strawberry','grapes']});
WriteResult({ "nInserted" : 1 })
> db.food.insert({_id:5,fruits:[ 'orange','grapes']});
WriteResult({ "nInserted" : 1 })
>
```

Outcome: Let us check if these documents are now in the "food" collection.

```
db.food.find({})
```

```
> db.food.find({})
{ "_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }
{ "_id" : 2, "fruits" : [ "orange", "butterfruit", "mango" ] }
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{ "_id" : 4, "fruits" : [ "banana", "strawberry", "grapes" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
>
```

Objective: To find those documents from the "food" collection which has the "fruits array" constituted of "banana", "apple" and "cherry".

Act:

```
db.food.find({fruits:['banana','apple','cherry']}).pretty()
```

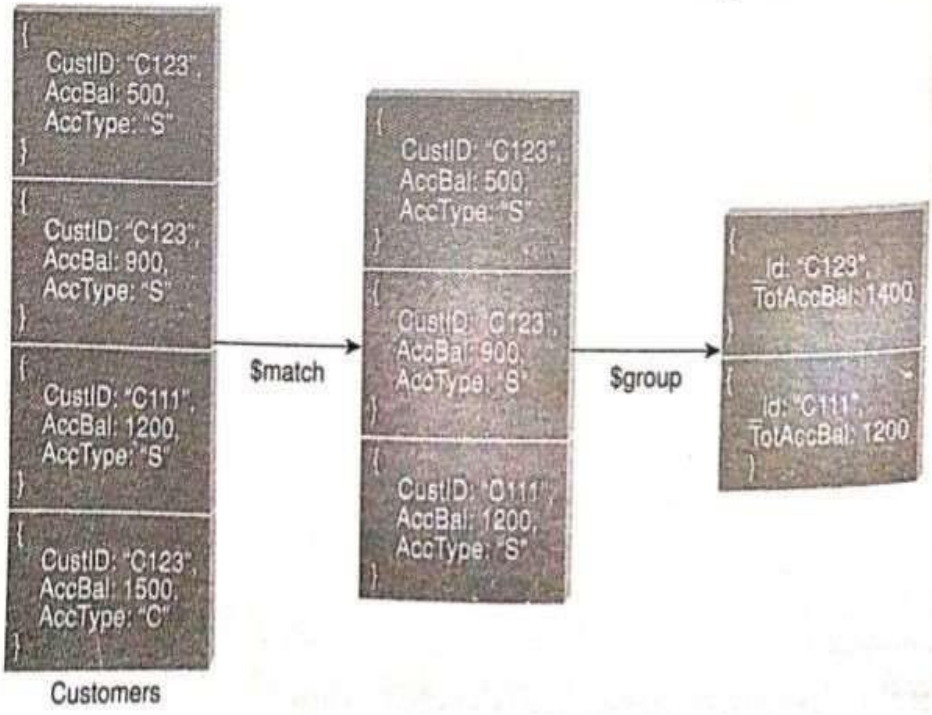
Outcome:

```
> db.food.find({fruits:['banana','apple','cherry']}).pretty()
{ "_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }
>
```

R.M.K GROUP OF INSTITUTIONS

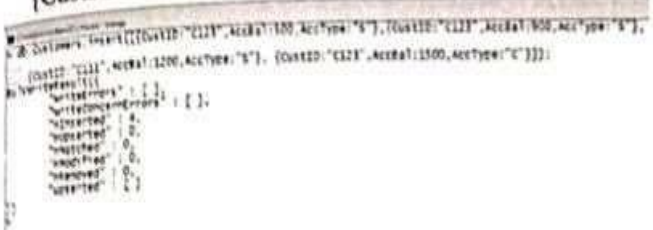
6.5.9 Aggregate Function

Objective: Consider the collection "Customers" as given below. It has four documents. We would like to filter out those documents where the "AccType" has a value other than "S". After the filter, we should be left with three documents where the "Acctype": "S". It is then required to group the documents on the basis of CustID and sum up the "AccBal" for each unique "CustID". This is similar to the output received with group by clause in RDBMS. Once the groups have been formed [as per the example below, there will be only two groups: (a) "CustID" : "C123" and (b) "CustID" : "C111"], filter and display that group where the "TotAccBal" column has a value greater than 1200.



Let us start off by creating the collection "Customers" with the above displayed four documents:

```
db.Customers.insert([{CustID:"C123",AccBal:500,AccType:"S"},
{CustID:"C123", AccBal: 900, AccType:"S"},
{CustID:"C111", AccBal: 1200, AccType:"S"},
{CustID:"C123", AccBal: 1500, AccType:"C"}]);
```



To confirm the presence of four documents in the "Customers" collection, use the below syntax:

```
db.Customers.find().pretty();
```

```
> db.Customers.find().pretty();
{
        "_id" : ObjectId("54993269f4263d015Cbfa72c"),
        "CustID" : "C123",
        "AccBal" : 500,
        "AccType" : "S"
}
{
        "_id" : ObjectId("54993269f4263d015Cbfa72d"),
        "CustID" : "C123",
        "AccBal" : 900,
        "AccType" : "S"
}
{
        "_id" : ObjectId("54993269f4263d015Cbfa72e"),
        "CustID" : "C111",
        "AccBal" : 1200,
        "AccType" : "S"
}
{
        "_id" : ObjectId("54993269f4263d015Cbfa72f"),
        "CustID" : "C123",
        "AccBal" : 1500,
        "AccType" : "C"
}
```

To group on "CustID" and compute the sum of "AccBal", use the below syntax:

```
db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } });
```

```
> db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } });
{ "_id" : "C111", "TotAccBal" : 1200 }
{ "_id" : "C123", "TotAccBal" : 2900 }
```

In order to first filter on "AccType:S" and then group it on "CustID" and then compute the sum of "AccBal", use the below syntax:

```
db.Customers.aggregate( { $match : {AccType : "S" } },
{ $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } });
```

```
> db.Customers.aggregate( { $match : {AccType : "S" } },
... { $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } });
{ "_id" : "C111", "TotAccBal" : 1200 }
{ "_id" : "C123", "TotAccBal" : 1400 }
```

In order to first filter on "AccType:S" and then group it on "CustID" and then to compute the sum of "AccBal" and then filter those documents wherein the "TotAccBal" is greater than 1200, use the below syntax:

```
db.Customers.aggregate( { $match : {AccType : "S" } },
{ $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } },
{ $match : {TotAccBal : { $gt : 1200 } }});
```

```
> db.Customers.aggregate( { $match : {AccType : "S" } },
... { $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } },
... { $match : {TotAccBal : { $gt : 1200 } }});
{ "_id" : "C123", "TotAccBal" : 1400 }
```

To group on "CustID" and compute the average of the "AccBal" for each group:

```
db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $avg : "$AccBal" } } });
```

```
> db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $avg : "$AccBal" } } });
{ "_id" : "C111", "TotAccBal" : 1200 }
{ "_id" : "C123", "TotAccBal" : 966.6666666666666 }
>
```

To group on "CustID" and determine the maximum "AccBal" for each group:

```
db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $max : "$AccBal" } } });
```

```
> db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $max : "$AccBal" } } });
{ "_id" : "C111", "TotAccBal" : 1200 }
{ "_id" : "C123", "TotAccBal" : 1500 }
>
```

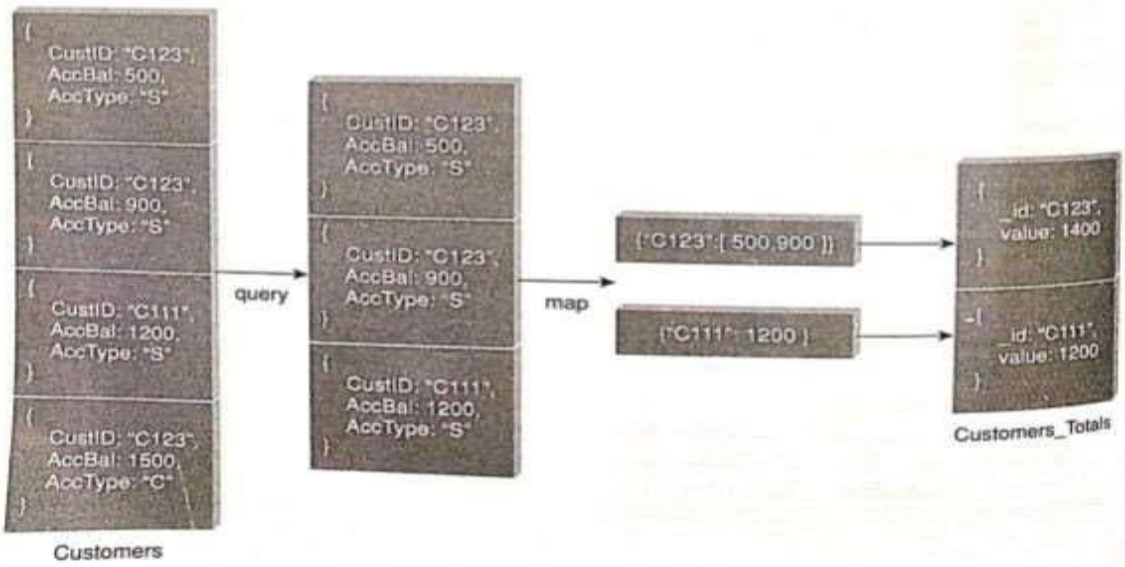
To group on "CustID" and determine the minimum "AccBal" for each group:

```
db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $min : "$AccBal" } } });
```

```
> db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $min : "$AccBal" } } });
{ "_id" : "C111", "TotAccBal" : 1200 }
{ "_id" : "C123", "TotAccBal" : 500 }
>
```

6.5.10 MapReduce Function

Objective: Consider the collection "Customers" below. There are four documents. Run a query to filter out those documents where the key "AccType" has a value other than "S". Then for each unique CustID, prepare a list of AccBal values. For example, for CustID: "C123", the AccBals are 500,900. This task will be assigned to the mapper function. The output from the mapper function serves as the input to the reducer function. The reducer function then aggregates the AccBal for each CustID. For example, for CustID: "C123", the value is 1400, etc.



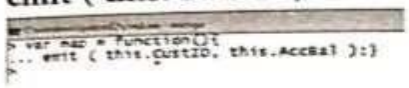
R.M.K GROUP OF INSTITUTIONS

Given below is the syntax that we will use to accomplish the objective.

```
db.Customers.mapReduce (
map        →    function() { emit ( this.CustID, this.AccBal ); },
reduce     →    function(key, values) { return Array.sum (values ) },
                {
query      →        query: { AccType: "S"},
output     →        out: "Customer_Totals"
                }
                )
```

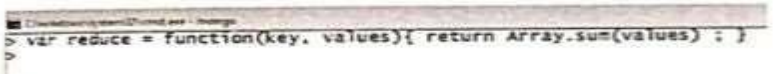
Map Function

```
var map = function(){
emit ( this.CustID, this.AccBal );}
```



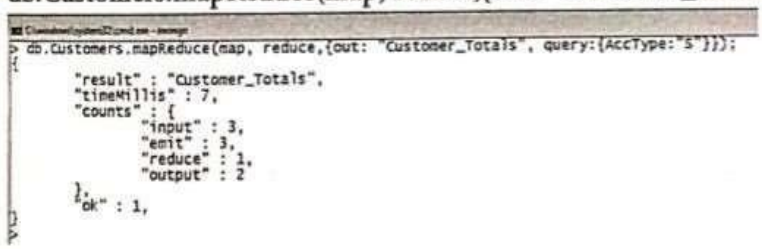
Reduce Function

```
var reduce = function(key, values){ return Array.sum(values) ; }
```



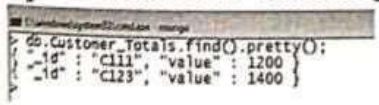
To execute the query

```
db.Customers.mapReduce(map, reduce,{out: "Customer_Totals", query:{AccType:"S"}});
```



```
> db.Customers.mapReduce(map, reduce,{out: "Customer_Totals", query:{AccType:"S"}});
{
        "result" : "Customer_Totals",
        "timeMillis" : 7,
        "counts" : {
                "input" : 3,
                "emit" : 3,
                "reduce" : 1,
                "output" : 2
        },
        "ok" : 1,
}
```

The output as archived in "Customer_Totals" collection:

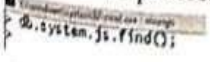


```
> db.Customer_Totals.find().pretty();
{ "_id" : "C111", "value" : 1200 }
{ "_id" : "C123", "value" : 1400 }
```

6.5.11 Java Script Programming

Objective: To compute the factorial of a given positive number. The user is required to create a function by the name "factorial" and insert it into the "system.js" collection.

Before we proceed, a quick check on what is contained in the "system.js" collection:



```
> db.system.js.find();
```

As per the screenshot above, currently there are no functions in the system.js collection.

Act:

```
db.system.js.insert({_id:"factorial",
```

R.M.K GROUP OF INSTITUTIONS

```
value:function(n)
{
        if (n==1)
        return 1;
        else
        return n * factorial(n-1);
}
        }
);
```

```
> db.system.js.insert({_id:"factorial",
... value:function(n)
... {
...  if (n==1)
... return 1;
...  else
...  return n * factorial(n-1);
... }
... });
WriteResult({ "nInserted" : 1 })
>
```

Confirm the presence of the "factorial" function in the system.js collection.

```
> db.system.js.find();
{ "_id" : "factorial", "value" : function (n)
{
if (n==1)
return 1;
else
return n * factorial(n-1);
} }
>
```

To execute the function "factorial", use the eval() method.

db.eval("factorial(3)");

```
> db.eval("factorial(3)");
6
>
```

db.eval("factorial(5)");

```
> db.eval("factorial(5)");
120
>
```

db.eval("factorial(1)");

```
> db.eval("factorial(1)");
1
>
```

6.5.12 Cursors in MongoDB

Objective: To create a collection by the name "alphabets" and insert documents in it containing two fields, "_id" and "alphabet". The values stored in the "alphabet" field should be "a", "b", "c", "d", etc. with one value stored per document. There should be 26 documents in all. We need to use cursor to iterate through the "alphabets" collection.

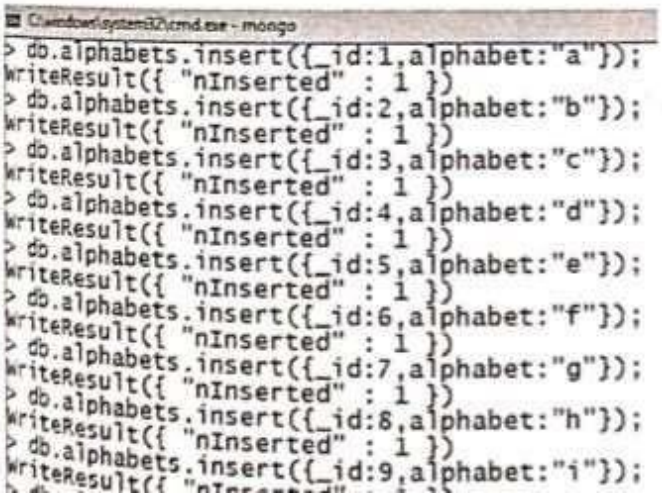
Note: "Alphabets" is the name of the collection and "alphabet" is the name of the field.

Act: To create the collection "alphabets" with its 26 documents.

```
db.alphabets.insert({_id:1,alphabet:"a"});
db.alphabets.insert({_id:2,alphabet:"b"});
```

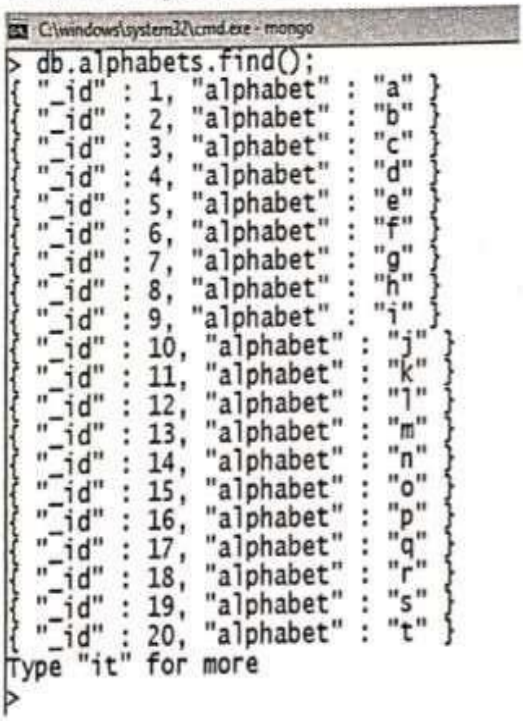
R.M.K GROUP OF INSTITUTIONS

```
db.alphabets.insert({_id:3,alphabet:"c"});
db.alphabets.insert({_id:4,alphabet:"d"});
db.alphabets.insert({_id:5,alphabet:"e"});
db.alphabets.insert({_id:6,alphabet:"f"});
db.alphabets.insert({_id:7,alphabet:"g"});
db.alphabets.insert({_id:8,alphabet:"h"});
db.alphabets.insert({_id:9,alphabet:"i"});
db.alphabets.insert({_id:10,alphabet:"j"});
db.alphabets.insert({_id:11,alphabet:"k"});
db.alphabets.insert({_id:12,alphabet:"l"});
db.alphabets.insert({_id:13,alphabet:"m"});
db.alphabets.insert({_id:14,alphabet:"n"});
db.alphabets.insert({_id:15,alphabet:"o"});
db.alphabets.insert({_id:16,alphabet:"p"});
db.alphabets.insert({_id:17,alphabet:"q"});
db.alphabets.insert({_id:18,alphabet:"r"});
db.alphabets.insert({_id:19,alphabet:"s"});
db.alphabets.insert({_id:20,alphabet:"t"});
db.alphabets.insert({_id:21,alphabet:"u"});
db.alphabets.insert({_id:22,alphabet:"v"});
db.alphabets.insert({_id:23,alphabet:"w"});
db.alphabets.insert({_id:24,alphabet:"x"});
db.alphabets.insert({_id:25,alphabet:"y"});
db.alphabets.insert({_id:26,alphabet:"z"});
```



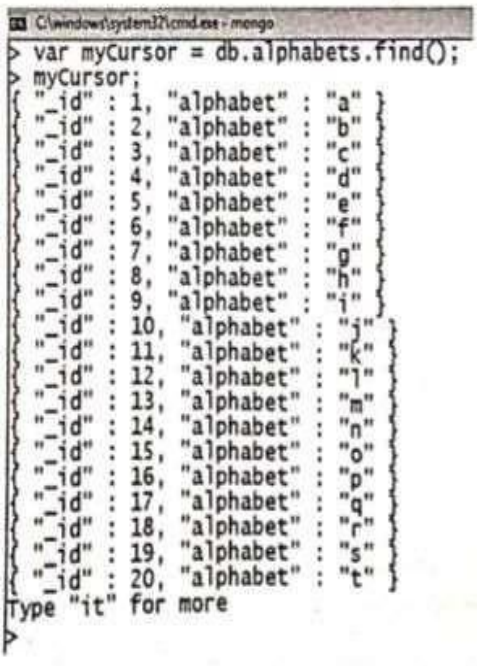
R.M.K GROUP OF INSTITUTIONS

Confirm the presence of 26 documents in the "alphabets" collection.



A quick word on how the db.collection.find() method works. This is the primary method for read operation. In other words, it allows one to fetch the documents from the collection. To be able to access the documents, one needs to iterate the cursor.

However, in the mongo shell, if the returned cursor is not assigned to a variable using the var keyword, then cursor is automatically iterated up to 20 times to print the first 20 documents in the result



R.M.K GROUP OF INSTITUTIONS

6.5.13 Indexes

Assume the collection with the following documents:

```
> db.books.find().pretty();
{
        "_id" : 6,
        "Category" : "Machine Learning",
        "Bookname" : "Machine Learning for Hackers",
        "Author" : "Drew Conway",
        "qty" : 25,
        "price" : 400,
        "rol" : 30,
        "pages" : 350
}
{
        "_id" : 7,
        "Category" : "Web Mining",
        "Bookname" : "Mining the Social Web",
        "Author" : "Matthew A.Russell",
        "qty" : 55,
        "price" : 500,
        "rol" : 30,
        "pages" : 250
}
{
        "_id" : 8,
        "Category" : "Python",
        "Bookname" : "Python for Data Analysis",
        "Author" : "Wes McKinney",
        "qty" : 8,
        "price" : 150,
        "rol" : 20,
        "pages" : 150
}
{
        "_id" : 9,
        "Category" : "Visualization",
        "Bookname" : "Visualizing Data",
        "Author" : "Ben Fry",
        "qty" : 12,
        "price" : 325,
        "rol" : 6,
        "pages" : 450
}
{
        "_id" : 10,
        "Category" : "Web Mining",
        "Bookname" : "Algorithms for the intelligent web",
        "Author" : "Haralambos Marmanis",
        "qty" : 5,
        "price" : 850,
        "rol" : 10,
        "pages" : 120
}
>
```

Create an index on the key "Category" in the "books" collection.

```
> db.books.ensureIndex({"Category":1});
{
        "createdCollectionAutomatically" : false,
        "numIndexesBefore" : 1,
        "numIndexesAfter" : 2,
        "ok" : 1
}
>
```

R.M.K GROUP OF INSTITUTIONS

Check on the status, that is, number and name of the indexes:

```
> db.books.stats();
{
        "ns" : "test.books",
        "count" : 5,
        "size" : 1200,
        "avgObjSize" : 240,
        "storageSize" : 8192,
        "numExtents" : 1,
        "nindexes" : 2,
        "lastExtentSize" : 8192,
        "paddingFactor" : 1,
        "systemFlags" : 1,
        "userFlags" : 1,
        "totalIndexSize" : 16352,
        "indexSizes" : {
                "_id_" : 8176,
                "Category_1" : 8176
        },
        "ok" : 1
}
>
```

Get the list of all indexes on the "books" collection:

```
> db.books.getIndexes();
[
        {
                "v" : 1,
                "key" : {
                        "_id" : 1
                },
                "name" : "_id_",
                "ns" : "test.books"
        },
        {
                "v" : 1,
                "key" : {
                        "Category" : 1
                },
                "name" : "Category_1",
                "ns" : "test.books"
        }
]
>
```

R.M.K GROUP OF INSTITUTIONS

To use the index on "Category" in the "books" collection, use the hint method:

```
> db.books.find({"Category":"Web Mining"}).pretty().hint({"Category":1});
{
        "_id" : 7,
        "Category" : "Web Mining",
        "Bookname" : "Mining the Social Web",
        "Author" : "Matthew A.Russell",
        "qty" : 55,
        "price" : 500,
        "rol" : 30,
        "pages" : 250
}
{
        "_id" : 10,
        "Category" : "Web Mining",
        "Bookname" : "Algorithms for the intelligent web",
        "Author" : "Haralambos Marmanis",
        "qty" : 5,
        "price" : 850,
        "rol" : 10,
        "pages" : 120
}
>
```

6.5.14 MongoImport

This command used at the command prompt imports CSV (Comma Separated Values) or TSV (Tab Separated Values) files or JSON (Java Script Object Notation) documents into MongoDB.

Objective: Given a CSV file "sample.txt" in the D: drive, import the file into the MongoDB collection, "SampleJSON". The collection is in the database "test".

The "sample.txt" file is as follows:

_id,FName,LName
1,Samuel,Jones
2,Virat,Kumar
3,Raul,"A Simpson"
4,,"Andrew Simon"

Act:

At the command prompt, execute the following command:

R.M.K GROUP OF INSTITUTIONS

Mongoimport --db test --collection SampleJSON --type csv --headerline --file d:\sample.txt

On successful execution of the command, the message at the prompt will be as follows:

```
connected to: 127.0.0.1
2015-02-20T21:09:27.301+0530 imported 4 objects
```

Output: To confirm the output, log into MongoDB shell and navigate to the "SampleJSON" collection in the "test" database.

The following are the JSON documents in the collection:

```
> db
test
> show collections
Customers
SampleJSON
books
fs.chunks
fs.files
persons
system.indexes
usercounters
users
> db.SampleJSON.find().pretty();
{ "_id" : 1, "FName" : "Samuel", "LName" : "Jones" }
{ "_id" : 2, "FName" : "Virat", "LName" : "Kumar" }
{ "_id" : 3, "FName" : "Raul", "LName" : "A Simpson" }
{ "_id" : 4, "FName" : "", "LName" : "Andrew Simon" }
>
```

6.5.15 MongoExport

This command used at the command prompt exports MongoDB JSON documents into CSV (Comma Separated Values) or TSV (Tab Separated Values) files or JSON (Java Script Object Notation) documents.

Objective: This command used at the command prompt exports MongoDB JSON documents from "Customers" collection in the "test" database into a CSV file "Output.txt" in the D: drive.

Given below is a snapshot of the JSON documents in the "Customers" collection of the "test" database.

```
> db
test
> show collections
Customers
SampleJSON
books
fs.chunks
fs.files
persons
system.indexes
usercounters
users
> db.Customers.find().pretty();
{
        "_id" : ObjectId("54df6d4f46a31d28183b9a5b"),
        "CustID" : "C123",
        "AccBal" : 500,
        "AccType" : "S"
}
{
        "_id" : ObjectId("54df6d4f46a31d28183b9a5c"),
        "CustID" : "C123",
        "AccBal" : 900,
        "AccType" : "S"
}
```

R.M.K GROUP OF INSTITUTIONS

```
{
        "_id" : ObjectId("54df6d4f46a31d28183b9a5d"),
        "CustID" : "C111",
        "AccBal" : 1200,
        "AccType" : "S"
}
{
        "_id" : ObjectId("54df6d4f46a31d28183b9a5e"),
        "CustID" : "C123",
        "AccBal" : 1500,
        "AccType" : "C"
}
>
```

Act: At the command prompt, execute the following command:

Mongoexport --db test --collection Customers --csv --fieldFile d:\fields.txt --out d:\output.txt

Before executing this command, ensure that you have created a "fields.txt" with a format defined as follows. The "fields.txt" file:

CustID
AccBal
AccType

For the MongoExport command to execute successfully, ensure that the fields are spelt as is in the MongoDB collection. The case also has to be maintained. It is mandatory to ensure that only one field name is placed per line.

On successful execution of the command, the message at the prompt will be as follows:

```
connected to: 127.0.0.1
exported 4 records
```

Output: To confirm the output, navigate to the D: drive and check the file "Output.txt".

"Output.txt"

```
CustID,AccBal,AccType
"C123",500.0,"S"
"C123",900.0,"S"
"C111",1200.0,"S"
"C123",1500.0,"C"
```

6.5.16 Automatic Generation of Unique Numbers for the "_id" Field

Step 1: Run the insert() method on a new collection "usercounters". This is to start off with an initial value of 0 for the "seq" field.

```
db.usercounters.insert(
{
_id: "empid",
seq:0
}
)
```

R.M.K GROUP OF INSTITUTIONS

Step 2: Create a user-defined function "getnextseq". This method will invoke "findAndModify()" method on the "usercounters" collection. This is to increment the value of seq field by 1 and update the same in "usercounters" collection.

```
function getnextseq(name) {
var ret=db.usercounters.findAndModify(
{
query: {_id:name},
update: {$inc:{seq:1}},
new:true
}
);
return ret.seq;
}
```

Step 3: Run the insert() method on the collection where you need to have the "_id" field and get the uniquely generated number. Notice the call to getnextseq() method as value to _id. The return value from the getnextseq() method becomes the value of _id.

```
db.users.insert(
{
_id:getnextseq("empid"),
Name: "sarah jane"
}
)
```

10. HIVE

Hive is a Data Warehousing tool that sits on top of Hadoop. Refer Figure 9.1. Hive is used to process structured data in Hadoop. The three main tasks performed by Apache Hive are:

1. Summarization
2. Querying
3. Analysis

Facebook initially created Hive component to manage their ever-growing volumes of log data. Later Apache software foundation developed it as open-source and it came to be known as Apache Hive.

R.M.K GROUP OF INSTITUTIONS

Hive makes use of the following

1. HDFS for Storage.
2. MapReduce for execution.
3. Stores metadata/schemas in an RDBMS.

- Hive provides HQL (Hive Query Language) or HiveQL which is similar to SQL.
- Hive compiles SQL queries into MapReduce jobs and then runs the job in the Hadoop Cluster.
- It is designed to Support OLAP (Online Analytical processing). Hive provides extensive data type functions and formats for data summarization and analysis.

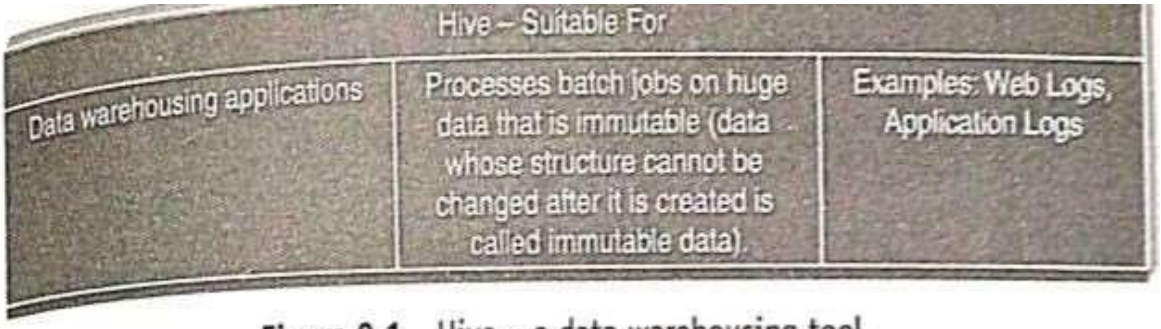


Figure 9.1 Hive – a data warehousing tool.

10.1 History of Hive and Recent Releases of Hive

The history of Hive and recent releases of Hive are illustrated pictorially in Figures 9.2 and 9.3, respectively



Figure 9.2 History of Hive.

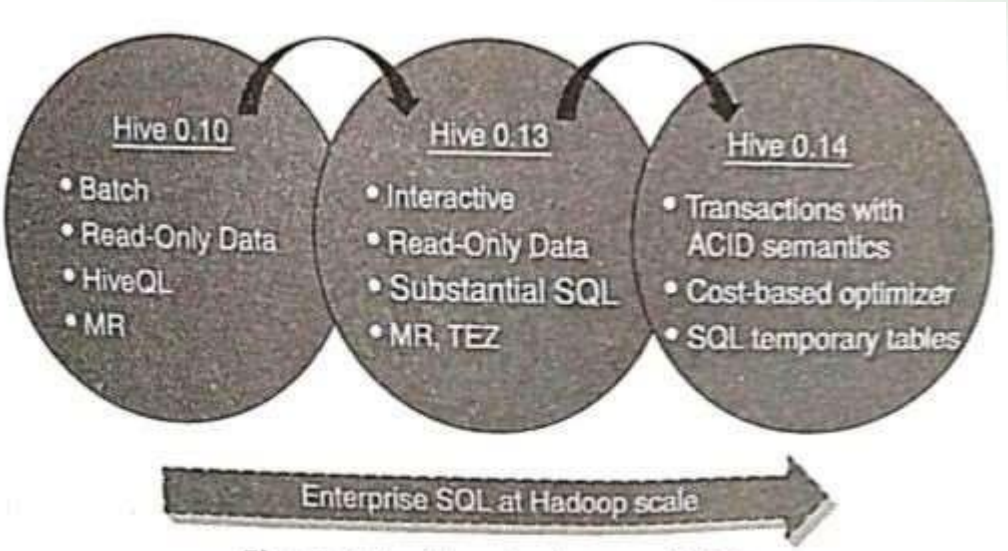


Figure 9.3 Recent releases of Hive.

2. Hive Features

1. It is similar to SQL.
2. HQL is easy
3. Hive supports rich data types such as structs, lists
4. Hive supports SQL Filters, group-by and order-by
5. Custom types, Custom Functions can be defined.

10.3 Hive Integration and Work Flow

Figure 9.4 depicts the flow of log file analysis. Explanation of the workflow: Hourly Log Data can be stored directly into HDFS and then data cleaning is performed on the log file. Finally, Hive table(s) can be created to query the log file.

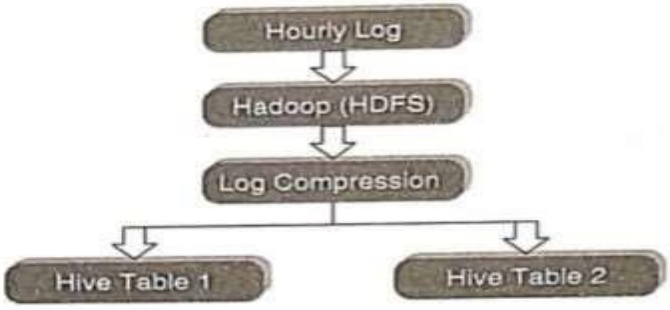


Figure 9.4 Flow of log analysis file.

10.4 Hive Data Units

1. **Databases:** The namespace for tables.
2. **Tables:** Set of records that have similar schema.
3. **Partitions:** Logical separations of data based on classification of given information as per specific attributes. Once hive has partitioned the data based on a specified key, it starts to assemble the records into specific folders as and when the records are inserted.
4. **Buckets (or Clusters):** Similar to partitions but uses hash function to segregate data and determines the cluster or bucket into which the record should be placed.

When to Use Partitioning/Bucketing?

- Bucketing works well when the field has high cardinality (cardinality is the number of values a column or field can have) and data is evenly distributed among
- Partitioning works best when the cardinality of the partitioning field is not too high. Partitioning can be done on multiple fields with an order (Year/Month/Day) whereas bucketing can be done on only one field.

Fig 9.5 shows how these data units are arranged in a Hive Cluster.

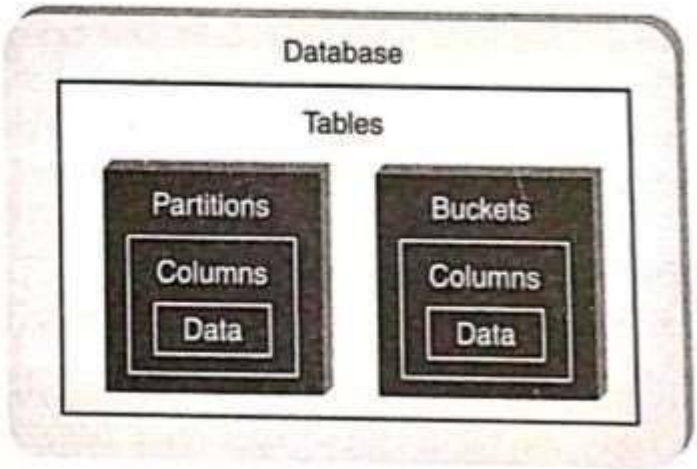


Figure 9.5 Data units as arranged in a Hive.

Fig 9.6 describes the semblance of Hive structure with database.

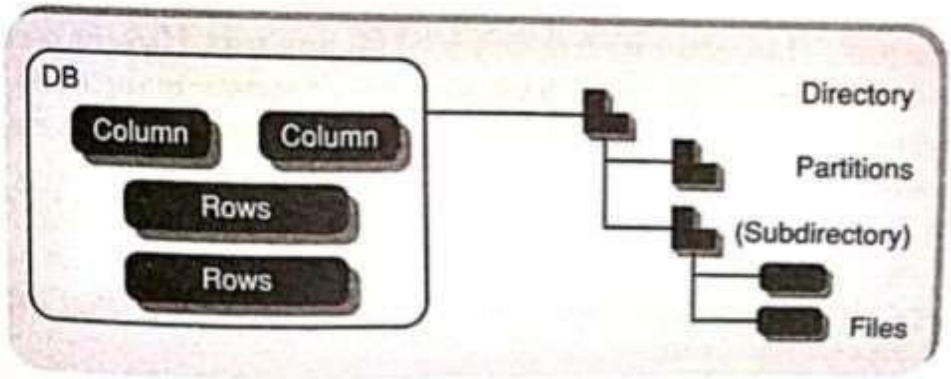


Figure 9.6 Semblance of Hive structure with database.

A database contains several tables. Each table is constituted of rows and columns. In Hive, tables are stored as a folder and partition tables are stored as a sub-directory. Bucketed tables are stored as a file.

10.5 HIVE ARCHITECTURE

Hive architecture is depicted in Figure 9.7. The various parts are as

1. **Hive Command-Line Interface (Hive CLI):** The most commonly used interface to interact with
2. **Hive Web interface:** It is a simple Graphic User Interface to interact with Hive and to execute
3. **Hive Server:** This is an optional server. This can be used to submit Hive Jobs from a remote client.
4. **JDBC/ODBC:** Jobs can be submitted from a JDBC Client. One can write a Java code to connect to Hive and submit jobs on it.
5. **Driver:** Hive queries are sent to the driver for compilation, optimization and execution.
6. **Metastore:** Hive table definitions and mappings to the data are stored in a Metastore.

R.M.K GROUP OF INSTITUTIONS

A Metastore consists of the following:

- Metastore service: Offers interface to the Hive.
- Database: Stores data definitions, mappings to the data and others.

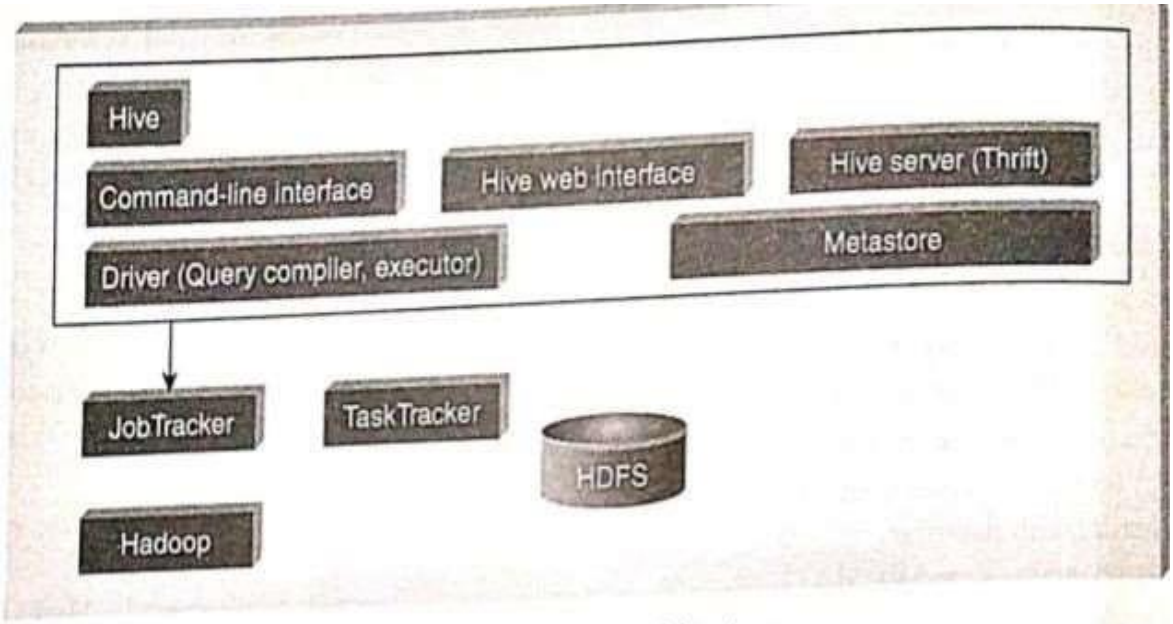


Figure 9.7 Hive architecture.

The metadata which is stored in the metastore includes IDs of Database, IDs of Tables, IDs of Indexes, etc., the time of creation of a Table, the Input Format used for a Table, the Output Format used for a Table, etc. The metastore is updated whenever a table is created or deleted from Hive. There are three kinds of metastore.

1. Embedded Metastore: This metastore is mainly used for unit tests. Here, only one process is allowed to connect to the metastore at a time. This is the default metastore for Hive. It is Apache Derby Database. In this metastore, both the database and the metastore service run embedded in the main Hive Server process. Figure 9.8 shows an Embedded Metastore.

R.M.K GROUP OF INSTITUTIONS

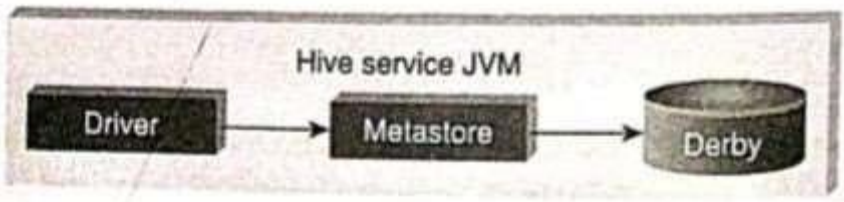


Figure 9.8 Embedded Metastore.

2. Local Metastore: Metadata can be stored in any RDBMS component like MySQL. Local metastore allows multiple connections at a time. In this mode, the Hive metastore service runs in the main Hive Server process, but the metastore database runs in a separate process, and can be on a separate host. Figure 9.9 shows a Local Metastore.

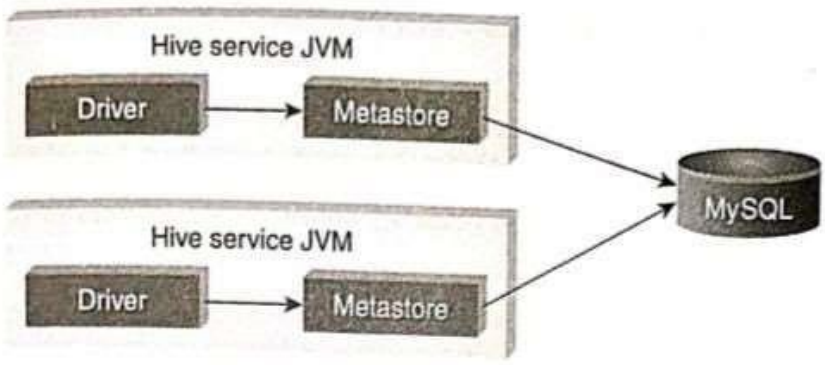


Figure 9.9 Local Metastore.

3. Remote Metastore: In this, the Hive driver and the metastore interface run on different JVMs (which can run on different machines as well) as in Figure 9.10. This way the database can be fire-walled from the Hive user and also database credentials are completely isolated from the users of Hive.

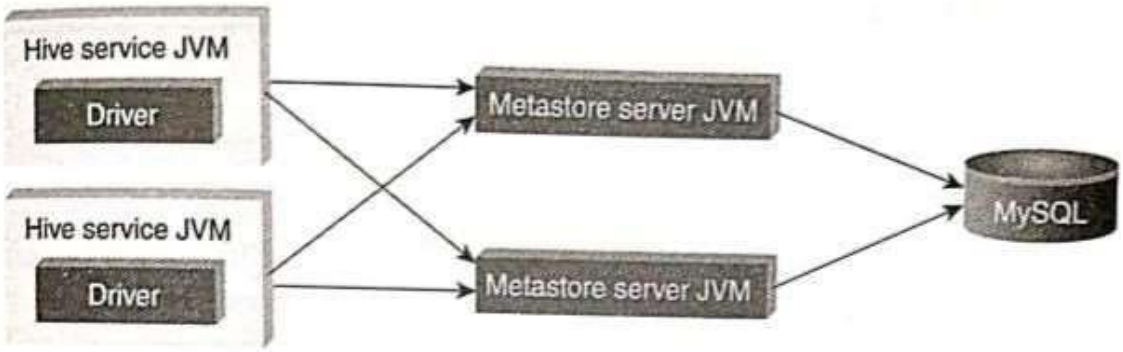


Figure 9.10 Remote Metastore.

R.M.K GROUP OF INSTITUTIONS

10.6 HIVE DATA TYPES

Primitive Data Types

Numeric Data Type	
TINYINT	1-byte signed integer
SMALLINT	2-byte signed integer
INT	4-byte signed integer
BIGINT	8-byte signed integer
FLOAT	4-byte single-precision floating-point
DOUBLE	8-byte double-precision floating-point number

String Types	
STRING	
VARCHAR	Only available starting with Hive 0.12.0
CHAR	Only available starting with Hive 0.13.0

Strings can be expressed in either single quotes (') or double quotes (")

Miscellaneous Types	
BOOLEAN	
BINARY	Only available starting with Hive

9.3.2 Collection Data Types

Collection Data Types	
STRUCT	Similar to 'C' struct. Fields are accessed using dot notation. E.g.: struct('John', 'Doe')
MAP	A collection of key-value pairs. Fields are accessed using [] notation. E.g.: map('first', 'John', 'last', 'Doe')
ARRAY	Ordered sequence of same types. Fields are accessed using array index. E.g.: array('John', 'Doe')

10.7 HIVE FILE FORMAT

The file formats in Hive specify how records are encoded in a file.

1. **Text File**
   - The default file format is text file. In this format, each record is a line in the file. In text file, different control characters are used as delimiters.
   - The delimiters are ^A (octal 001, separates all fields), ^B (octal 002, separates the elements in the array or struct), ^C (octal 003, separates key-value pair), and \n.
   - The term **field** is used when overriding the default delimiter. The supported text files are CSV and TSV. JSON or XML documents too can be specified as text file.

2. **Sequential File**
   - Sequential files are flat files that store binary key-value
   - It includes compression support which reduces the CPU, I/O requirement.

3. **RCFile (Record**
   - RCFile stores the data in **Column Oriented Manner** which ensures that **Aggregation** operation is not an expensive
   - For example, consider a table which contains four columns as shown in Table 9.1.
   - Instead of only partitioning the table horizontally like the row-oriented DBMS (row-store), RCFile partitions this table first horizontally and then vertically to serialize the data.
   - Based on the user-specified value, first the table is partitioned into multiple row groups horizontally.

R.M.K GROUP OF INSTITUTIONS

- Depicted in Table 9.2, Table 9.1 is partitioned into two row groups by considering three rows as the size of each row group.
- Next, in every row group RCFile partitions the data vertically like column-store. So the table will be serialized as shown in Table 9.3.

Table 9.1 A table with four columns

C1	C2	C3	C4
11	12	13	14
21	22	23	24
31	32	33	34
41	42	43	44
51	52	53	54

Table 9.2 Table with two row groups

Row Group 1			
C1	C2	C3	C4
11	12	13	14
21	22	23	24
31	32	33	34

Row Group 2			
C1	C2	C3	C4
41	42	43	44
51	52	53	54

Table 9.3 Table in RCFile Format

Row Group 1	Row Group 2
11, 21, 31;	41, 51;
12, 22, 32;	42, 52;
13, 23, 33;	43, 53;
14, 24, 34;	44, 54;

10.8 HIVE QUERY LANGUAGE (HQL)

Hive query language provides basic SQL like operations. Here are few of the tasks which HQL can do easily.

1. Create and manage tables and partitions.
2. Support various Relational, Arithmetic, and Logical Operators.
3. Evaluate functions.
4. Download the contents of a table to a local directory or result of queries to HDFS directory.

10.8.1 DDL (Data Definition Language) Statements

These statements are used to build and modify the tables and other objects in the database. The DDL commands are as follows:

1. Create/Drop/Alter Database
2. Create/Drop/Truncate
3. Alter
4. Create/Drop/Alter
5. Create/Drop/Alter Index
6. Show
7. Describe

R.M.K GROUP OF INSTITUTIONS

2. DML (Data Manipulation Language) Statements

These statements are used to retrieve, store, modify, delete, and update data in database. The DML Commands are as follows:

1. Loading files into table.
2. Inserting data into Hive Tables from queries.

R.M.K GROUP OF INSTITUTIONS

9.5.3 Starting Hive Shell

To start Hive, go to the installation path of Hive and type as below:

```
[root@volgalnx005 ~]# hive

Logging initialized using configuration in jar:file:/root/Desktop/VMDATA/Hive/hi
ve/lib/hive-common-0.14.0.jar!/hive-log4j.properties
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/root/Desktop/VMDATA/Hadoop/hadoop/share/hadoo
p/common/lib/slf4j-log4j12-1.7.5.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/root/Desktop/VMDATA/Hive/hive/lib/hive-jdbc-0
.14.0-standalone.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
hive>
```

The sections have been designed as follows:

Objective:	What is it that we are trying to achieve here?
Input (optional):	What is the input that has been given to us to act upon?
Act:	The actual statement/command to accomplish the task at hand.
Outcome:	The result/output as a consequence of executing the statement.

9.5.4 Database

A database is like a container for data. It has a collection of tables which houses the data.

Objective: To create a database named "STUDENTS" with comments and database properties.

Act:

```
CREATE DATABASE IF NOT EXISTS STUDENTS COMMENT 'STUDENT Details'
WITH DBPROPERTIES ('creator' = 'JOHN');
```

Outcome:

```
hive> CREATE DATABASE IF NOT EXISTS STUDENTS COMMENT 'STUDENT Details' WITH DBPROPERTIES ('creato
r' = 'JOHN');
OK
Time taken: 0.536 seconds
hive>
```

Explanation of the syntax:

IF NOT EXIST: It is an optional clause. The create database statement with "IF Not EXISTS" clause creates a database if it does not exist. However, if the database already exists then it will notify the user that a database with the same name already exists and will not show any error message.

COMMENT: This is to provide short description about the database.

WITH DBPROPERTIES: It is an optional clause. It is used to specify any properties of database in the form of (key, value) separated pairs. In the above example, "Creator" is the "Key" and "JOHN" is the value.

We can use "**SCHEMA**" in place of "**DATABASE**" in this command.

Note: We have not specified the location where the Hive database will be created. By default all the Hive databases will be created under default warehouse directory (set by the property hive.metastore.warehouse.dir) as /user/hive/warehouse/database_name.db. But if we want to specify our own location, then the LOCATION clause can be specified. This clause is optional.

Objective: To display a list of all databases.

Act:

```
SHOW DATABASES;
```

Outcome:

```
hive> SHOW DATABASES;
OK
students
Time taken: 0.082 seconds, Fetched: 22 row(s)
hive>
```

By default, **SHOW DATABASES** lists all the databases available in the metastore. We can use "**SCHEMAS**" in place of "**DATABASES**" in this command. The command has an optional "Like" clause. It can be used to filter the database names using regular expressions such as "*" , "?", etc.

SHOW DATABASES LIKE "Stu*"

SHOW DATABASES like "Stud???s"

Objective: To describe a database.

Act:

```
DESCRIBE DATABASE STUDENTS;
```

Note: Shows only DB name, comment, and DB directory.

Outcome:

```
hive> DESCRIBE DATABASE STUDENTS;
OK
students        STUDENT Details hdfs://volgalnx010.ad.infosys.com:9000/user/hive/warehouse/studen
ts.db   root    USER
Time taken: 0.03 seconds, Fetched: 1 row(s)
hive>
```

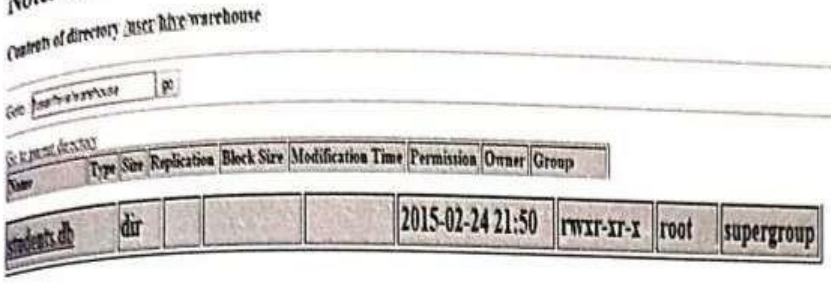
R.M.K GROUP OF INSTITUTIONS

Objective: To drop database.

Act:

```
DROP DATABASE STUDENTS;
```

Note: Hive creates database in the warehouse directory of Hive as shown below:



Now assume that the database "STUDENTS" has 10 tables within it. How do we delete the complete database along with the tables contained therein?

Use the command:

```
DROP DATABASE STUDENTS CASCADE;
```

By default the mode is RESTRICT which implies that the database will NOT be dropped if it contains tables.

Note: The complete syntax is as follows:

```
DROP DATABASE [IF EXISTS] database_name [RESTRICT | CASCADE]
```

9.5.5 Tables

Hive provides two kinds of table:

1. Internal or Managed Table
2. External Table

9.5.5.1 *Managed Table*

1. Hive stores the Managed tables under the warehouse folder under Hive.
2. The complete life cycle of table and data is managed by Hive.
3. When the internal table is dropped, it drops the data as well as the metadata.

When you create a table in Hive, by default it is internal or managed table. If one needs to create an external table, one will have to use the keyword "EXTERNAL".

R.M.K GROUP OF INSTITUTIONS

Objective: To create managed table named 'STUDENT'.

Act:

```
CREATE TABLE IF NOT EXISTS STUDENT(rollno INT,name STRING,gpa FLOAT) ROW
FORMAT DELIMITED FIELDS TERMINATED BY '\t';
```

Outcome:

```
hive> CREATE TABLE IF NOT EXISTS STUDENT(rollno INT,name STRING,gpa FLOAT) ROW FORMAT DELIMITED F
IELDS TERMINATED BY '\t';
OK
Time taken: 0.355 seconds
hive>
```

Objective: To describe the "STUDENT" table.

Act:

```
DESCRIBE STUDENT;
```

Outcome:

```
hive> DESCRIBE STUDENT;
OK
rollno                  int
name                    string
gpa                     float
Time taken: 0.163 seconds, Fetched: 3 row(s)
hive>
```

Note: Hive creates managed table in the warehouse directory of Hive as shown below:

Contents of directory /user/hive/warehouse/students.db

Goto : /user/hive/warehouse/student go

Go to parent directory

Name	Type	Size	Replication	Block Size	Modification Time	Permission	Owner	Group
student	dir				2015-02-24 22:03	rwxr-xr-x	root	supergroup

Go back to DFS home

Local logs

Log directory

Hadoop, 2015.

R.M.K GROUP OF INSTITUTIONS

To check whether an existing table is managed or external, use the below syntax:

DESCRIBE FORMATTED tablename;

It displays complete metadata of a table. You will see one row called table type which will display either MANAGED_TABLE OR EXTERNAL_TABLE.

DESCRIBE FORMATTED STUDENT;

9.5.5.2 External or Self-Managed Table

1. When the table is dropped, it retains the data in the underlying location.
2. **External** keyword is used to create an external table.
3. **Location** needs to be specified to store the dataset in that particular location.

R.M.K

Objective: To create external table named 'EXT_STUDENT'.

Act:

```
CREATE EXTERNAL TABLE IF NOT EXISTS EXT_STUDENT(rollno INT,name
STRING,gpa FLOAT) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'
LOCATION '/STUDENT_INFO;
```

Outcome:

```
hive> CREATE EXTERNAL TABLE IF NOT EXISTS EXT_STUDENT(rollno INT,name STRING,gpa FLOAT) ROW FORMA
T DELIMITED FIELDS TERMINATED BY '\t'  LOCATION '/STUDENT_INFO';
OK
Time taken: 0.123 seconds
hive>
```

Note: Hive creates the external table in the specified location.

R.M.K GROUP OF INSTITUTIONS

9.5.5.3 Loading Data into Table from File

Objective: To load data into the table from file named student.tsv.

Act:

```
LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' OVERWRITE INTO TABLE
EXT_STUDENT;
```

Note: Local keyword is used to load the data from the local file system. In this case, the file is copied. To load the data from HDFS, remove local key word from the statement. In this case, the file is moved from the original location.

Outcome:

```
hive> LOAD DATA LOCAL INPATH  '/root/hivedemos/student.tsv' OVERWRITE INTO TABLE EXT_STUDENT;
Loading data to table students.ext_student
Table students.ext_student stats: [numFiles=0, numRows=0, totalSize=0, rawDataSize=0]
OK
Time taken: 5.034 seconds
hive>
```

9.5.5.4 Collection Data Types

Objective: To work with collection data types.

Input:

1001,John,Smith:Jones,Mark1!45:Mark2!46:Mark3!43
1002,Jack,Smith:Jones,Mark1!46:Mark2!47:Mark3!42

Act:

```
CREATE TABLE STUDENT_INFO (rollno INT,name String, sub ARRAY<STRING>,marks
MAP<STRING,INT>)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
COLLECTION ITEMS TERMINATED BY ':'
MAP KEYS TERMINATED BY '!';
LOAD DATA LOCAL INPATH '/root/hivedemos/studentinfo.csv' INTO TABLE
STUDENT_INFO;
```

Outcome:

```
hive> CREATE TABLE STUDENT_INFO (rollno INT,name String, sub ARRAY<STRING>,marks MAP<STRING,FLOAT>)
    > ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
    > COLLECTION ITEMS TERMINATED BY ':'
    > MAP KEYS TERMINATED BY '!';
OK
Time taken: 0.112 seconds
hive>
```

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/studentinfo.csv' INTO TABLE STUDENT_INFO;
Loading data to table students.student_info
Table students.student_info stats: [numFiles=1, totalSize=109]
OK
Time taken: 0.397 seconds
hive>
```

9.5.5.5 Querying Table

Objective: To retrieve the student details from "EXT_STUDENT" table.

Act:

```
SELECT * from EXT_STUDENT;
```

Outcome:

```
hive> select * from EXT_STUDENT;
OK
1001    John    3.0
1002    Jack    4.0
1003    Smith   4.5
1004    Scott   4.2
1005    Joshi   3.5
1006    Alex    4.5
1007    David   4.2
1008    James   4.0
1009    John    3.0
1010    Joshi   3.5
Time taken: 0.054 seconds, Fetched: 10 row(s)
hive>
```

Objective: Querying Collection Data Types.

Act:

```
SELECT * from STUDENT_INFO;
SELECT NAME,SUB FROM STUDENT_INFO;
// To retrieve value of Mark1
SELECT NAME, MARKS['Mark1'] from STUDENT_INFO;
// To retrieve subordinate (array) value
SELECT NAME,SUB[0] FROM STUDENT_INFO;
```

Outcome:

```
hive> SELECT * from STUDENT_INFO;
OK
1001    John    ["Smith","Jones"]       {"Mark1":45,"Mark2":46,"Mark3":43}
1002    Jack    ["Smith","Jones"]       {"Mark1":46,"Mark2":47,"Mark3":42}
Time taken: 0.044 seconds, Fetched: 2 row(s)
```

```
hive> SELECT NAME,SUB FROM STUDENT_INFO;
OK
John    ["Smith","Jones"]
Jack    ["Smith","Jones"]
Time taken: 0.061 seconds, Fetched: 2 row(s)
hive>
```

```
hive> SELECT NAME, MARKS['Mark1']  from STUDENT_INFO;
OK
John    45
Jack    46
Time taken: 0.06 seconds, Fetched: 2 row(s)
hive>
```

```
hive> SELECT NAME,SUB[0] FROM STUDENT_INFO;
OK
John    Smith
Jack    Smith
Time taken: 0.071 seconds, Fetched: 2 row(s)
hive>
```

R.M.K GROUP OF INSTITUTIONS

10.8.6 Partitions

- In Hive, the query reads the entire dataset even though a where clause filter is specified on a particular column.
- This becomes a bottleneck in most of the MapReduce jobs as it involves huge degree of I/O. So it is necessary to reduce I/O required by the MapReduce job to improve the performance of the query.
- A very common method to reduce I/O is data partitioning. Partitions split the larger dataset into more meaningful chunks.

We will try to understand partitioning with the help of a simple example.

PICTURE THIS...

"XYZ enterprise" has a wide customer base spread across several states in the US. The data has been fed to the central system. The senior leadership team would like to get a report providing the statewise Sales %.

The IT team has proposed two options to help service this request:

1. Run the query with the where clause of each state name (such as where StateName = "A"). This will mean that the entire dataset is scanned/read through for each and every state. If we have data for 25 states, the dataset is read 25 times (once for each state). Assuming we have 5 million records, it would mean that 5 million records are read 25 times. This can be a major performance bottle neck and will worsen as the dataset grows.
2. The second option stated here can help alleviate this problem. The IT team can start off with creating 25 folders (one for each state) and instruct to have the data pertaining to states place into the folder of the respective states. This arrangement will greatly help at the time of querying the data. To get the Sales % of a particular state, the folder belonging to that state only has to be scanned/read through.

This intelligent way of grouping data during data load is termed as PARTITIONING in Hive.

This brings us to the next question.

If you are looking through the folder for state = "A", is there any possibility of you coming across data for state = "B" or state = "C"? Think through! I am sure your answer will be No! And we know the reason.

A point to note here is that as we create the partitions, there is no need to include the partitioned column along with the other columns of the dataset as this is something that is automatically taken care of "BY PARTITIONED" clause.

In our example above, we will refrain from adding the partitioned column along with other columns or the dataset and trust Hive to automatically manage this.

Partition is of two types:

1. STATIC PARTITION: It is upon the user to mention the partition (the segregation unit) where data from the file is to be loaded.

2. DYNAMIC PARTITION: The user is required to simply state the column, basis which the partitioning will take place.

Hive will then create partitions basis the unique values in the column on which partition is to be carried out.

9.5.6.1 Static Partition

Static partitions comprise columns whose values are known at compile time.

Objective: To create static partition based on "gpa" column.

Act:

```
CREATE TABLE IF NOT EXISTS STATIC_PART_STUDENT (rollno INT, name STRING)
PARTITIONED BY (gpa FLOAT) ROW FORMAT DELIMITED FIELDS TERMINATED
BY '\t';
```

Outcome:

```
hive> CREATE TABLE IF NOT EXISTS STATIC_PART_STUDENT(rollno INT,name STRING) PARTITIONED BY (gpa FLOAT) RO
W FORMAT DELIMITED FIELDS TERMINATED BY '\t';
OK
Time taken: 0.105 seconds
hive>
```

Objective: Load data into partition table from table.

Act:

```
INSERT OVERWRITE TABLE STATIC_PART_STUDENT PARTITION (gpa =4.0)
SELECT rollno, name from EXT_STUDENT where gpa=4.0;
```

Outcome:

```
hive> INSERT OVERWRITE TABLE STATIC_PART_STUDENT PARTITION (gpa =4.0) SELECT rollno,name from EXT_STUDENT
where gpa=4.0;
Query ID = root_20150224230404_4500d58a-cb21-4912-ba40-788e5cf8f9da
Total jobs = 3
```

Objective: To add one more static partition based on "gpa" column using the "alter" statement.

Act:

```
ALTER TABLE STATIC_PART_STUDENT ADD PARTITION (gpa=3.5);
INSERT OVERWRITE TABLE STATIC_PART_STUDENT PARTITION (gpa =4.0) SELECT
rollno,name from EXT_STUDENT where gpa=4.0;
```

Outcome:

```
hive> ALTER TABLE STATIC_PART_STUDENT ADD PARTITION (gpa=3.5);
OK
Time taken: 0.166 seconds
hive>
```

Contents of directory /user/hive/warehouse/students.db/static_part_student

Goto : /user/hive/warehouse/student go

Go to parent directory

Name	Type	Size	Replication	Block Size	Modification Time	Permission	Owner	Group
gpa=3.5	dir				2015-02-24 23:09	rwxr-xr-x	root	supergroup
gpa=4.0	dir				2015-02-24 23:11	rwxr-xr-x	root	supergroup

Go back to DFS home

9.5.6.2 *Dynamic Partition*

Dynamic partition have columns whose values are known only at Execution Time.

Objective: To create dynamic partition on column date.

Act:

```
CREATE TABLE IF NOT EXISTS DYNAMIC_PART_STUDENT(rollno INT,name STRING)
PARTITIONED BY (gpa FLOAT) ROW FORMAT DELIMITED FIELDS TERMINATED
BY '\t';
```

Objective: To load data into a dynamic partition table from table.

Act:

```
SET hive.exec.dynamic.partition = true;
SET hive.exec.dynamic.partition.mode = nonstrict;
```

Note: The dynamic partition strict mode requires at least one static partition column. To turn this off, set hive.exec.dynamic.partition.mode=nonstrict

```
INSERT OVERWRITE TABLE DYNAMIC_PART_STUDENT PARTITION (gpa) SELECT
rollno,name,gpa from EXT_STUDENT;
```

Outcome:

Contents of directory /user/hive/warehouse/students.db/dynamic_part_student

Goto : /user/hive/warehouse/student go

Go to parent directory

Name	Type	Size	Replication	Block Size	Modification Time	Permission	Owner	Group
gpa=3.0	dir				2015-02-24 23:16	rwxr-xr-x	root	supergroup
gpa=3.5	dir				2015-02-24 23:16	rwxr-xr-x	root	supergroup
gpa=4.0	dir				2015-02-24 23:16	rwxr-xr-x	root	supergroup
gpa=4.2	dir				2015-02-24 23:16	rwxr-xr-x	root	supergroup
gpa=4.3	dir				2015-02-24 23:16	rwxr-xr-x	root	supergroup

Go back to DFS home

Note: Create partition for all values.

9.5.7 Bucketing

Bucketing is similar to partition. However, there is a subtle difference between partition and bucketing. In a partition, you need to create partition for each unique value of the column. This may lead to situations where you may end up with thousands of partitions. This can be avoided by using Bucketing in which you can limit the number of buckets that will be created. A bucket is a file whereas a partition is a directory.

Objective: To learn the concept of bucket in hive.

Act:

```
CREATE TABLE IF NOT EXISTS STUDENT (rollno INT,name STRING,grade FLOAT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' INTO TABLE STUDENT;
```

Set below property to enable bucketing.

```
set hive.enforce.bucketing=true;
```

```
// To create a bucketed table having 3 buckets
CREATE TABLE IF NOT EXISTS STUDENT_BUCKET (rollno INT,name STRING,grade
FLOAT)
CLUSTERED BY (grade) into 3 buckets;
// Load data to bucketed table
FROM STUDENT
INSERT OVERWRITE TABLE STUDENT_BUCKET
SELECT rollno,name,grade;
// To display content of first bucket
SELECT DISTINCT GRADE FROM STUDENT_BUCKET
TABLESAMPLE(BUCKET 1 OUT OF 3 ON GRADE);
```

Outcome:

```
hive> CREATE TABLE IF NOT EXISTS STUDENT (rollno INT,name STRING,grade FLOAT)
    > ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
OK
Time taken: 0.101 seconds
hive>
```

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' INTO TABLE STUDENT;
Loading data to table book.student
Table book.student stats: [numFiles=1, totalSize=145]
OK
Time taken: 0.536 seconds
hive>
```

```
hive> set hive.enforce.bucketing=true;
hive>
```

R.M.K GROUP OF INSTITUTIONS

```
hive> CREATE TABLE IF NOT EXISTS STUDENT_BUCKET (rollno INT,name STRING,grade FLOAT)
    > CLUSTERED BY (grade) into 3 buckets;
OK
Time taken: 0.101 seconds
hive>
```

```
hive> FROM STUDENT
    > INSERT OVERWRITE TABLE STUDENT_BUCKET
    > SELECT rollno,name,grade;
```

3 buckets have been created as shown below:

Contents of directory /user/hive/warehouse/book.db/student_bucket

Goto

Go to parent directory

Name	Type	Size	Replication	Block size	Modification Time	Permission	Owner	Group
000000_0	file	59 B	3	128 MB	2015-03-10 22:29	rw-r--r--	root	supergroup
000001_0	file	59 B	3	128 MB	2015-03-10 22:29	rw-r--r--	root	supergroup
000002_0	file	28 B	3	128 MB	2015-03-10 22:29	rw-r--r--	root	supergroup

Go back to DFS home

Local logs

Log directory

Hadoop, 2015.

9.5.8 Views

In Hive, view support is available only in version starting from 0.6. Views are purely logical object.

Objective: To create a view table named "STUDENT_VIEW".

Act:

```
CREATE VIEW STUDENT_VIEW AS SELECT rollno, name FROM EXT_STUDENT;
```

Outcome:

```
hive> CREATE VIEW STUDENT_VIEW AS SELECT rollno,name FROM EXT_STUDENT;
OK
Time taken: 0.606 seconds
hive>
```

R.M.K GROUP OF INSTITUTIONS

Objective: Querying the view "STUDENT_VIEW".

Act:

```
SELECT * FROM STUDENT_VIEW LIMIT 4;
```

Outcome:

```
hive> SELECT * FROM STUDENT_VIEW LIMIT 4;
OK
1001    John
1002    Jack
1003    Smith
1004    Scott
Time taken: 0.279 seconds, Fetched: 4 row(s)
hive>
```

Objective: To drop the view "STUDENT_VIEW".

Act:

```
DROP VIEW STUDENT_VIEW;
```

Outcome:

```
hive> DROP VIEW STUDENT_VIEW;
OK
Time taken: 0.452 seconds
hive>
```

9.5.9 Sub-Query

In Hive, sub-queries are supported only in the FROM clause (Hive 0.12). You need to specify name for sub-query because every table in a FROM clause has a name. The columns in the sub-query select list should have unique names. The columns in the subquery select list are available to the outer query just like columns of a table.

Objective: Write a sub-query to count occurrence of similar words in the file.

Act:

```
CREATE TABLE docs (line STRING);
LOAD DATA LOCAL INPATH '/root/hivedemos/lines.txt' OVERWRITE INTO TABLE docs;
CREATE TABLE word_count AS
SELECT word, count(1) AS count FROM
(SELECT explode (split (line, ' ')) AS word FROM docs) w
GROUP BY word
ORDER BY word;
SELECT * FROM word_count;
```

Outcome:

```
hive> CREATE TABLE docs (line STRING);
OK
Time taken: 0.118 seconds
hive>
```

R.M.K GROUP OF INSTITUTIONS

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/lines.txt' OVERWRITE INTO TABLE docs;
Loading data to table students.docs
Table students.docs stats: [numFiles=1, numRows=0, totalSize=91, rawDataSize=0]
OK
Time taken: 2.697 seconds
hive>
```

```
hive> CREATE TABLE word_count AS
    > SELECT word, count(1) AS count FROM
    > (SELECT explode (split (line, ' ')) AS word FROM docs) w
    > GROUP BY word
    > ORDER BY word;
```

```
hive> SELECT * FROM word_count;
OK
Hadoop  2
Hive    2
Introducing     1
Introduction    1
Pig     1
Session 3
Welcome 1
to      2
Time taken: 0.062 seconds, Fetched: 8 row(s)
hive>
```

Note: The explode() function takes an array as input and outputs the elements of the array as separate rows.

In Hive 0.13, sub-queries are supported in the where clause as well.

9.5.10 Joins

Joins in Hive is similar to the SQL Join.

Objective: To create JOIN between Student and Department tables where we use RollNo from both the tables as the join key.

Act:

```
CREATE TABLE IF NOT EXISTS STUDENT(rollno INT,name STRING,gpa FLOAT) ROW
FORMAT DELIMITED FIELDS TERMINATED BY '\t';

LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' OVERWRITE INTO TABLE
STUDENT;

CREATE TABLE IF NOT EXISTS DEPARTMENT(rollno INT,deptno int,name STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
```

R.M.K GROUP OF INSTITUTIONS

LOAD DATA LOCAL INPATH '/root/hivedemos/department.tsv' OVERWRITE INTO TABLE DEPARTMENT;

SELECT a.rollno, a.name, a.gpa, b.deptno FROM STUDENT a JOIN DEPARTMENT b ON a.rollno = b.rollno;

Outcome:

```
hive> CREATE TABLE IF NOT EXISTS STUDENT(rollno INT,name STRING,gpa FLOAT) ROW FORMAT D
ELIMITED FIELDS TERMINATED BY '\t';
OK
Time taken: 0.115 seconds
hive>
```

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' OVERWRITE INTO TABLE STUDENT
;
Loading data to table students.student
Table students.student stats: [numFiles=1, numRows=0, totalSize=145, rawDataSize=0]
OK
Time taken: 0.723 seconds
hive>
```

```
hive> CREATE TABLE IF NOT EXISTS DEPARTMENT(rollno INT,deptno int,name STRING) ROW FORM
AT DELIMITED FIELDS TERMINATED BY '\t';
OK
Time taken: 0.099 seconds
hive>
```

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/department.tsv' OVERWRITE INTO TABLE DEPA
RTMENT;
Loading data to table students.department
Table students.department stats: [numFiles=1, numRows=0, totalSize=120, rawDataSize=0]
OK
Time taken: 0.442 seconds
hive>
```

```
hive> SELECT a.rollno, a.name, a.gpa, b.deptno  FROM STUDENT  a JOIN DEPARTMENT b ON a.
rollno = b.rollno;
OK
1001
1002    John    3.0     101
1003    Jack    4.0     102
1004    Smith   4.5     103
1005    Scott   4.2     104
1006    Joshi   3.5     105
1007    Alex    4.5     101
1008    David   4.2     104
```

9.5.11 Aggregation

Hive supports aggregation functions like avg, count, etc.

Objective: To write the average and count aggregation functions.

Act:

```
SELECT avg(gpa) FROM STUDENT;
SELECT count(*) FROM STUDENT;
```

Outcome:

```
hive> SELECT avg(gpa) FROM STUDENT;

OK
3.839999961853027
Time taken: 28.639 seconds, Fetched: 1 row(s)
hive>

hive> SELECT count(*) FROM STUDENT;

OK
10
Time taken: 26.218 seconds, Fetched: 1 row(s)
hive>
```

9.5.12 Group By and Having

Data in a column or columns can be grouped on the basis of values contained therein by using "Group By". "Having" clause is used to filter out groups NOT meeting the specified condition.

Objective: To write group by and having function.

Act:

```
SELECT rollno, name,gpa FROM STUDENT GROUP BY rollno,name,gpa HAVING gpa >
4.0;
```

Outcome:

```
1003    Smith   4.5
1004    Scott   4.2
1006    Alex    4.5
1007    David   4.2
Time taken: 78.972 seconds, Fetched: 4 row(s)
hive>
```

9.6 RCFILE IMPLEMENTATION

RCFile (Record Columnar File) is a data placement structure that determines how to store relational tables on computer clusters.

Objective: To work with RCFILE Format.

Act:

```
CREATE TABLE STUDENT_RC( rollno int, name string,gpa float ) STORED AS RCFILE;
INSERT OVERWRITE table STUDENT_RC SELECT * FROM STUDENT;
SELECT SUM(gpa) FROM STUDENT_RC;
```

Outcome:

```
hive> CREATE TABLE STUDENT_RC( rollno int, name string,gpa float ) STORED AS RCFILE;
OK
Time taken: 0.093 seconds
hive>
```

```
hive> INSERT OVERWRITE table STUDENT_RC SELECT * from STUDENT;
```

```
hive> SELECT SUM(gpa) from STUDENT_RC;
```

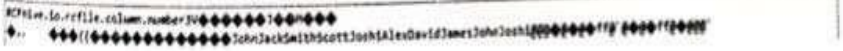
```
OK
38.39999961853027
Time taken: 25.41 seconds, Fetched: 1 row(s)
hive>
```

Note: Stores the data in column oriented manner.

File: /user/hive/warehouse/students.db/student_rc/000000_0

Goto: /user/hive/warehouse/student go

Go back to dir listing
Advanced view/download options



R.M.K GROUP OF INSTITUTIONS

9.8 USER-DEFINED FUNCTION (UDF)

In Hive, you can use custom functions by defining the User-Defined Function (UDF).

Objective: Write a Hive function to convert the values of a field to uppercase.

Act:

```
package com.example.hive.udf;
import org.apache.hadoop.hive.ql.exec.Description;
import org.apache.hadoop.hive.ql.exec.UDF;
@Description(
 name="SimpleUDFExample")
public final class MyLowerCase extends UDF {
 public String evaluate(final String word) {
 return word.toLowerCase();
 }
}
```

Note: Convert this Java Program into Jar.

```
ADD JAR /root/hivedemos/UpperCase.jar;
CREATE TEMPORARY FUNCTION touppercase AS 'com.example.hive.udf.MyUpperCase';
SELECT TOUPPERCASE(name) FROM STUDENT;
```

Outcome:

```
hive> ADD JAR /root/hivedemos/UpperCase.jar;
Added [/root/hivedemos/UpperCase.jar] to class path
Added resources: [/root/hivedemos/UpperCase.jar]
hive> CREATE TEMPORARY FUNCTION touppercase AS 'com.example.hive.udf.MyUpperCase';
OK
Time taken: 0.014 seconds
hive>
```

```
hive> Select touppercase (name) from STUDENT;
OK
JOHN
JACK
SMITH
SCOTT
JOSHI
ALEX
DAVID
JAMES
JOHN
JOSHI
Time taken: 0.061 seconds, Fetched: 10 row(s)
hive>
```

R.M.K
GROUP OF
INSTITUTIONS

VIDEO LINKS

Unit – III

R.M.K
GROUP OF
INSTITUTIONS

VIDEO LINKS

Sl. No.	Topic	Video Link
1	EDA fundamentals	https://www.youtube.com/watch?v=-o3AxdVcU_tQ&t=55s
2	Introduction to NoSQL	https://www.youtube.com/watch?v=XFE0EgT5o_QE
3	RDBMS Vs MongoDB	https://www.youtube.com/watch?v=ZS_kXvOeQ_5Y
4	MongoDB Query Language	https://www.youtube.com/watch?v=11phu1pEJ_m4
5	Hive Architecture	https://www.youtube.com/watch?v=rr17cbPGW_GA

GROUP OF INSTITUTIONS

10. ASSIGNMENT : UNIT – III

Easy Level:

1. During the data preprocessing step, how should one treat missing/null values? How will you deal with them?
2. What is the lifecycle of the data science project?
3. How can the data be normalized?
4. What is the difference between the save and insert commands in MongoDB, and when do they act similarly?

Medium Level:

1. Mention what are the components of a Hive query processor?
2. Describe MongoDB and its key features. How does it differ from traditional RDBMS in terms of data storage and retrieval?
3. Provide examples of basic CRUD (Create, Read, Update, Delete) operations in MongoDB using its query language.

Hard Level:

1. Provide examples of basic HiveQL queries to perform SELECT, JOIN, and AGGREGATION operations.
2. What is an RC (Record Columnar) file in Hive? Discuss its advantages and use cases.
3. Choose one EDA tool and provide a step-by-step guide on how to perform a basic EDA using this tool.

R.M.K GROUP OF INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

PART-AQ&A UNIT-III

R.M.K
GROUP OF
INSTITUTIONS

11. PART A : Q & A : UNIT – III

1. What is EDA ? (CO3 , K2)
   - EDA is a process of examining the available dataset to discover patterns, spot anomalies, test hypotheses, and check assumptions using statistical measures.
   - The primary aim of EDA is to examine what data can tell us before actually going through formal modeling or hypothesis formulation.
   - Exploratory data analysis is the stage where we actually start to understand the message contained in the data.

2. What is the Significance of EDA ? (CO3 , K2)
   - Exploratory data analysis is key, and usually the first exercise in data mining. It allows us to visualize data to understand it as well as to create hypotheses for further
   - EDA actually reveals ground truth about the content without making any underlying
   - Key components of exploratory data analysis include summarizing data, statistical analysis, and visualization of data.

3. Mention the steps involved in EDA. (CO3 , K2)
   - ➢ Problem definition
   - ➢ Data preparation
   - ➢ Data analysis
   - ➢ Development and representation of the results

4. What do you mean by numerical data? (CO3 , K2)
   - This data has a sense of measurement involved in it; for example, a person's age, height, weight, blood pressure, heart rate, temperature, number of teeth, number of bones, and the number of family members.

R.M.K GROUP OF INSTITUTIONS

- This data is often referred to as quantitative data in statistics.
- The numerical dataset can be either discrete or continuous types.

5. What is categorical data ? (CO3 , K2)

➢ This type of data represents the characteristics of an object; for example, gender, marital status, type of address, or categories of the movies.

➢ This data is often referred to as qualitative datasets in statistics.

6. What are the different types of categorical variables? (CO3 , K2)

- A binary categorical variable can take exactly two values and is also referred to as a **dichotomous variable**. For example, when we create an experiment, the result is either success or failure. Hence, results can be understood as a **binary categorical variable**.
- Polytomous variables are categorical variables that can take more than two possible values.

7. What are the software tools available for EDA? (CO3 , K2

There are several software tools that are available to facilitate EDA. Some of the open source tools are:

- **Python:** This is an open source programming language widely used in data analysis, data mining, and data science.
- **R programming language:** R is an open source programming language that is widely utilized in statistical computation and graphical data analysis.
- **Weka:** This is an open source data mining package that involves several EDA tools and algorithms.
- **KNIME:** This is an open source tool for data analysis and is based on Eclipse.

11. PART A : Q & A : UNIT – III

8. What is the function used to identify NaN values ? (CO3 , K2)

- The isnull() function from the pandas library is used to identify NaN values. The true values in the dataframe indicate the values that are NaN.
- Alternatively, we can also use the notnull() method to do the same thing. The only difference would be that the function will indicate True for the values which are not null.

9. What will be the output of how = 'all' argument? (CO3 , K2)

- We can drop rows that have NaN values. To do so, we can use the how=all argument to drop only those rows whose values are entirely NaN: dfx.dropna(how='all')
- In the output, the entire rows containing NaN values will be removed.

10. What is the use of rename() method ? (CO3 , K2)

- If we want to create a transformed version of the dataframe, then we can use the rename()
- This method is handy when we do not want to modify the original data.

11. What do you mean by Discretization and binning ? (CO3 , K2)

Often when working with continuous datasets, we need to convert them into discrete or interval forms. Each interval is referred to as a bin, and hence the name binning.

12. How do you perform random sampling without replacement? (CO3 , K2)

To compute random sampling without replacement, we follow these steps:

1. Create a permutation array.
2. Slice off the first n elements of the array where n is the desired size of the subset we want to sample.
3. Use the df.take() method to obtain actual samples

R.M.K GROUP OF INSTITUTIONS

11. PART A : Q & A : UNIT – III

13. What are the features of NoSQL databases ? (CO3 , K2)

1. They are open source.
2. They are non-relational.
3. They are distributed.
4. They are schema-less.
5. They are cluster friendly.

14. What are types of NoSQL databases ? (CO3 , K2)

NoSQL databases are non-relational. They can be broadly classified into the following:

1. Key-value or the big hash table.
2. Schema-less.

15. What do you mean by NewSQL? (CO3 , K2)

- We need a database that has the same scalable performance of NoSQL systems for On Line Transaction Processing (OLTP) while still maintaining the ACID guarantees of a traditional
- This new modern RDBMS is called
- It supports relational data model and uses SQL as their primary interface.

16. What is MongoDB ? (CO3 , K2)

MongoDB is

1. Cross-platform.
2. Open source.
3. Non-relational.
4. Distributed.
5. NoSQL.
6. Document-oriented data store

11. PART A : Q & A : UNIT – III

17. What is CRUD operation in MongoDB ? (CO3 , K2)

CRUD (Create, Read Update, and Delete) operations in MongoDB

Create -> Creation of data is done using insert() or update() or save() method.

Read -> Reading the data is performed using the find() method.

Update -> Update to data is accomplished using the update() method with UPSERT set to false

Delete -> a document is Deleted using the remove() method.

18. What are the three main tasks performed by Apache Hive? (CO3 , K2)

The three main tasks performed by Apache Hive are:

1. Summarization
2. Querying
3. Analysis

19. What are the features of Hive? (CO3 , K2)

1. It is similar to
2. HQL is easy to
3. Hive supports rich data types such as structs, lists and
4. Hive supports SQL Filters, group-by and order-by
5. clauses.Custom types, Custom Functions can be defined.

20. What is Hive metastore? (CO3 , K2)

Hive table definitions and mappings to the data are stored in a Metastore. A Metastore consists of the following:

- Metastore service: Offers interface to the Hive.
- Database: Stores data definitions, mappings to the data and others.

There are three kinds of metastore.

i. Embedded Metastore

ii. Local Metastore

iii. Remote Metastore

R.M.K GROUP OF INSTITUTIONS

12. PART B QUESTIONS : UNIT – III

1. Explain the phases of Data analysis in detail. (CO3, K2)
2. Describe the steps involved in Exploratory Data analysis. (CO3, K2)
3. Write short notes on Numerical and Categorical data. (CO3, K2)
4. How does EDA differ from classical and Bayesian analysis? (CO3, K2)
5. Explain the concept of handling missing data. (CO3, K2)
6. What is NoSQL? Explain its types and advantages. (CO3, K2)
7. Explain the difference between SQL and NoSQL. (CO3, K2)
8. What is MongoDB? How do you create and drop the MongoDB Databases? (CO3, K2)
9. Write short notes on MongoDB Query Language. (CO3, K2)
10. What is Hive? Explain the Hive architecture in detail. (CO3, K2)
11. Explain Hive data types and file formats. (CO3, K2)
12. Write short notes on Hive Query Language. (CO3, K2)
13. Define Exploratory Data Analysis (EDA). Discuss its significance in the data science workflow
14. Describe the main objectives of EDA. How does EDA differ from confirmatory data analysis?
15. Identify and describe three software tools commonly used for EDA. Discuss the strengths and weaknesses of each.

R.M.K GROUP OF INSTITUTIONS

13. SUPPORTIVE ONLINE CERTIFICATION COURSES

NPTEL : https://onlinecourses.nptel.ac.in/noc21_cs45/preview

coursera : https://www.coursera.org/professional-certificates/google-data-analytics

Udemy : https://www.udemy.com/topic/data-analysis/

Mooc : https://www.mooc-list.com/tags/data-analytics

edx : https://www.edx.org/learn/data-analytics

R.M.K GROUP OF INSTITUTIONS

14. REAL TIME APPLICATIONS

Exploratory data analytics has applications in many fields, such as:

- **Professional sports:** Sports Analysts rely on EDA to search out the most successful players and teams, as well as to discover the variables that contribute to a team's wins and losses.
- The insights gathered from sports data also can help those involved with online sports betting at places such as DraftKings to make more informed bets. In addition, EDA is a helpful tool for deciding which players or teams should be selected for a company to endorse.

- **History:** EDA can be applied to create new data about past events. By using data collected from sources such as archeological digs, digitized photos, and text, it's possible for Data Analysts to have a more robust understanding of past events that have remained a mystery for millennia.
- **Healthcare:** EDA is helpful for spotting natural patterns embedded in large stores of medical data. In addition, healthcare networks, health departments, and hospitals store large amounts of data in electronic medical records (EMRs). Although there are strict compliance regulations in place to ensure patients' privacy, those working within healthcare seek out new ways to leverage this data without having to link it to specific individuals.
- By creating data mining systems to test EMRs, it's possible to gain important insights into the progression of chronic diseases such as kidney disease.

- **Marketing:** EDA sheds insights into various purchasing circumstances, such as why customers are no longer buying a product or why a particular campaign is successful. By understanding the context surrounding those variables, EDA provides Analysts with a strategy for future action.

- **The hospitality industry:** One of the main variables that affect those working in the hospitality industry is the substantial number of booking cancellations that occur.
- Of the estimated 140 million yearly bookings made online, approximately 40% of these reservations are canceled, leading to revenue loss and complications with distribution management.
- In order to predict which guests are most likely to cancel their reservations, EDA tools like machine learning and Python can be applied to a dataset to provide information about the features that contribute most to cancellations.
- This allows those working in hospitality to predict future cancellations in order to devise ways to reduce these numbers, as well as to establish safeguards such as cancellation fees for those who are most likely to cancel.

- **Retail:** EDA can be used by business managers to spot weak areas in a store or franchise in order to suggest areas that can be targeted for increased revenue.

- **Fraud detection:** When EDA data mining techniques are used on Medicare datasets, it's possible to evaluate the risk of a given individual for fraudulent activity.

- **Auditing:** EDA can be applied to several stages of auditing, for both internal and external audit cycles.

- **Geography:** Exploratory spatial data analysis (ESDA) is a branch of EDA that is concerned specifically with geographical data. Those with training in this field can perform a variety of geographical tasks, such as visualizing spatial distributions, spotting physical outliers, and uncovering spatial clusters or patterns.

15. CONTENTS BEYOND SYLLABUS : UNIT – III

Apache Cassandra

Apache Cassandra is an open source NoSQL distributed database trusted by thousands of companies for scalability and high availability without compromising performance.

- Linear scalability and proven fault-tolerance on commodity hardware or cloud infrastructure make it the perfect platform for mission-critical data.
- By design, NoSQL databases are lightweight, open-source, non-relational, and largely distributed.
- Counted among their strengths are horizontal scalability, distributed architectures, and a flexible approach to schema definition.

NoSQL databases enable rapid, ad-hoc organization and analysis of extremely high-volume, disparate data types. That's become more important in recent years, with the advent of Big Data and the need to rapidly scale databases in the cloud.

Cassandra is among the NoSQL databases that have addressed the constraints of previous data management technologies, such as SQL databases.

One important Cassandra attribute is that its databases are distributed. That yields both technical and business advantages.

Cassandra databases easily scale when an application is under high stress, and the distribution also prevents data loss from any given datacenter's hardware failure.

A distributed architecture also brings technical power; for example, a developer can tweak the throughput of read queries or write queries in isolation.

R.M.K GROUP OF INSTITUTIONS

Assessment Schedule (Proposed Date & Actual Date)

R.M.K GROUP OF INSTITUTIONS

16. Assessment Schedule (Proposed Date & Actual Date)

Sl. No.	ASSESSMENT	Proposed Date	Actual Date
1	FIRST INTERNAL ASSESSMENT		
2	SECOND INTERNAL ASSESSMENT		
3	MODEL EXAMINATION		
4	END SEMESTER EXAMINATION		

R.M.K GROUP OF INSTITUTIONS

17. PRESCRIBED TEXT BOOKS & REFERENCE BOOKS

TEXT BOOKS:

1. Subhashini Chellappan, Seema Acharya, "Big Data and Analytics", 2nd edition, Wiley Publications, 2019.
2. Suresh Kumar Mukhiya and Usman Ahmed, "Hands-on Exploratory Data Analysis with Python", Packt publishing, March 2020.
3. Jure Leskovek, Anand Rajaraman and Jefrey Ullman," Mining of Massive Datasets. v2.1", Cambridge University Press,2019.
4. Glenn J. Myatt, Wayne P. Johnson, Making Sense of Data II : A Practical Guide To Data Visualization, Advanced Data Mining Methods, and Applications, Wiley 2009.

REFERENCES:

1. Nelli, F., Python Data Analytics: with Pandas, NumPy and Matplotlib, Apress, 2018.
2. Bart Baesens," Analytics in a Big Data World: The Essential Guide to Data Science and its Applications", John Wiley & Sons, 2014
3. Min Chen, Shiwen Mao, Yin Zhang, Victor CM Leung, Big Data: Related Technologies, Challenges and Future Prospects, Springer, 2014.
4. Michael Minelli, Michele Chambers, Ambiga Dhiraj, "Big Data, Big Analytics: Emerging Business Intelligence and Analytic Trends", John Wiley & Sons, 2013.
5. Marcello Trovati, Richard Hill, Ashiq Anjum, Shao Ying Zhu, "Big Data Analytics and cloud computing – Theory, Algorithms and Applications", Springer International Publishing, 2016

18. MINI PROJECT SUGGESTION

Mini Project 1: Exploratory Data Analysis (EDA) on a Public Dataset
Objective: Perform EDA on a public dataset (e.g., Titanic dataset, Iris dataset, etc.) to gain insights.

Mini Project 2: NoSQL Database Implementation with MongoDB
Objective: Implement a NoSQL database using MongoDB and perform basic operations.

Mini Project 3: Data Analysis with Hive
Objective: Use Hive for data analysis on a large dataset stored in Hadoop.

Mini Project 4: Comparative Analysis of EDA Techniques
Objective: Compare EDA techniques with classical and Bayesian analysis methods on a chosen dataset.

Mini Project 5: Data Transformation and Visualization Pipeline
Objective: Create a pipeline to transform and visualize data from a NoSQL database.

R.M.K GROUP OF INSTITUTIONS

Thank you

Disclaimer:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

R.M.K GROUP OF INSTITUTIONS